

- NeoGeo中文几何描述 DSL 规范 v1.8.8
 - 目录
 - 1. 前言
 - 2. DSL 设计原则
 - 2.1 一条 DSL = 一个基本几何事实 (Atomic Fact)
 - 2.2 所有关键字必须小写
 - 2.3 点名必须为大写字母 (或带 prime 后缀)
 - 2.4 线段用两个点名连写表示
 - 2.5 与中文描述结构对齐
 - 2.6 简单核心 + 可扩展宏
 - 3. 形式化语法定义 (EBNF)
 - 4. 基础元素 (Entities)
 - 4.1 点的声明
 - 4.2 线的声明
 - 4.3 圆的声明
 - 4.4 基本图形声明
 - 5. 几何关系 (Relations)
 - 5.1 平行与垂直
 - 5.2 等长关系
 - 5.3 角度关系
 - 5.4 共线与共圆
 - 5.5 全等与相似
 - 5.6 点的位置关系
 - 5.7 切线关系
 - 5.8 角平分线关系
 - 6. 操作型构造指令 (Operations)
 - 6.1 连接
 - 6.2 延长
 - 6.3 作垂线
 - 6.4 作平行线
 - 6.5 作角平分线
 - 6.6 作高
 - 6.7 作中线
 - 7. 图形结构 (Shapes)
 - 7.1 基本多边形
 - 7.1.1 正多边形声明
 - 7.2 特殊三角形 (宏指令)

- 7.2.1 等腰三角形
 - 7.2.2 等腰直角三角形
 - 7.2.3 等边三角形
 - 7.2.4 直角三角形
- 7.3 特殊四边形
 - 7.3.1 矩形
 - 7.3.2 平行四边形
 - 7.3.3 梯形 (v1.8.3)
 - 7.3.4 正方形
- 8. 宏指令与展开规则 (Macros)
 - 8.0 命名与参数设计原则
 - 8.1 中线
 - 8.2 高
 - 8.3 角平分线
 - 8.4 垂直平分线约束
 - 8.4.1 点在垂直平分线上
 - 8.4.2 线段是中垂线 (v1.8.2)
 - 8.5 特殊三角形宏展开表
 - 8.6 旋转至目标 (rotate_to) (v1.8.4)
- 9. 圆与弧 (Circles & Arcs)
 - 9.1 圆的声明
 - 9.2 点与圆的关系
 - 9.3 弧
 - 9.4 扇形声明 (v1.8.0)
 - 9.5 内接与外接
 - 9.6 半圆
 - 9.7 三角形四心 (v1.8.8)
- 10. 角度与长度表达
 - 10.1 角度
 - 10.2 线段长度
 - 10.2.1 数学表达式支持 (v1.6.9)
 - 支持的表达式类型
 - 使用示例
 - 适用范围
 - 注意事项
 - 符号约定
 - 对应中文表达
 - 实现细节

- 10.3 平行线段间距离 (v1.8.5)
- 10.4 比例关系
 - 10.4.1 角度比例约束 (v1.6.7)
 - 10.4 角度单位约定 (v1.6)
 - 10.5 角度的三角函数值约束 (v1.6.4)
- 11. 点的位置关系 (高级)
 - 11.1 中点
 - 11.2 三等分点
 - 11.3 垂足
 - 11.4 交点
 - 11.5 弧中点 (v1.6.6)
 - 11.6 延长线上的点
- 12. 几何变换与折叠操作
 - 12.1 基本几何变换
 - 12.2 折叠操作
- 13. 坐标系与函数图像 (Coordinate & Functions)
 - 13.1 坐标系声明
 - 13.2 坐标点声明
 - 13.3 基础函数图像声明
 - 13.3.1 一次函数与二次函数
 - 13.3.2 幂函数 (Power Functions)
 - 13.3.3 反比例函数
 - 13.3.4 指数函数 (Exponential Functions)
 - 13.3.5 对数函数 (Logarithmic Functions)
 - 13.3.6 三角函数 (Trigonometric Functions)
 - 13.3.7 反三角函数 (Inverse Trigonometric Functions)
 - 13.4 通用函数表达式
 - 13.5 函数变换
 - 13.6 函数与几何的交互
 - 13.7 坐标系中的几何图形
 - 13.8 中文表达对照 (坐标系与函数相关)
 - 13.8.1 函数点坐标约束 (v1.7.0)
- 14. 模糊语义消歧指令 (Disambiguation)
 - 14.1 设计动机
 - 14.2 线段方向与位置指令
 - 14.2.1 方向指示 direction
 - 14.2.2 延长线方向 on_extended
 - 14.2.3 射线声明 ray

- 14.3 区域内外关系
 - 14.3.1 内部 inside
 - 14.3.2 外部 outside
 - 14.3.3 扇形区域
 - 14.3.4 半平面 half_plane
 - 14.3.5 异侧/同侧约束 opposite_sides / same_sides (v1.8.6)
 - 14.3.6 阴影区域标记 shaded
- 14.4 角平分线消歧 on(P, bisector, near=...)
- 14.5 任选点 choose
 - 14.5.1 线上任选点
 - 14.5.2 区域内任选点
- 14.6 消歧指令的 Extractor 处理
- 14.7 中文表达对照（消歧相关）
- 15. 中文表达对照表
 - 15.1 基础图形描述
 - 15.2 点与位置关系
 - 15.3 线段与角度关系
 - 15.4 操作型表达
 - 15.5 特殊三角形
 - 15.6 可忽略的表达
- 16. 合法性规则（Validity Rules）
 - 类型检查规则
 - 线段与直线的语义区分
- 17. 标准化与规范化（Canonicalization）(v1.6)
 - 17.1 DSL 语句拓扑排序
 - 17.1.1 Layer 2（图形声明）vs Layer 7（几何关系）的区别
 - 17.2 Operations 与 Macros 的展开
 - 17.2.1 等价写法归一化
 - 17.3 点名标准化
- 18. 标注示例
 - 18.1 示例 1：圆周角定理
 - 18.2 示例 2：矩形折叠问题
 - 18.3 示例 3：平行四边形与垂足
 - 18.4 示例 4：等腰三角形与中线
 - 18.5 示例 5：作图操作
- 19. 版本管理与变更日志
 - 19.1 版本号规则
 - 19.2 向后兼容性承诺

- 19.3 变更日志
 - [v1.8.8 \(2026-02-28\)](#)
 - [v1.8.7 \(2026-03-03\)](#)
 - [v1.8.6 \(2026-03-03\)](#)
 - [v1.8.5 \(2026-03-02\)](#)
 - [v1.8.4 \(2026-02-28\)](#)
 - [v1.8.3 \(2026-01-28\)](#)
 - [v1.8.2 \(2026-01-28\)](#)
 - [v1.8.1 \(2026-01-28\)](#)
 - [v1.8.0 \(2026-01-28\)](#)
 - [v1.7.1 \(2026-01-25\)](#)
 - [v1.7.0 \(2026-01-24\)](#)
 - [v1.6.9 \(2026-01-24\)](#)
 - [v1.6.8 \(2026-01-24\)](#)
 - [v1.6.7 \(2026-01-23\)](#)
 - [v1.6.6 \(2026-01-23\)](#)
 - [v1.6.6 \(2026-01-20\)](#)
 - [v1.6.5 \(2026-01-22\)](#)
 - [v1.6.4 \(2026-01-21\)](#)
 - [v1.6.3 \(2026-01-20\)](#)
 - [v1.6.2 \(2026-01-16\)](#)
 - [v1.6.1 \(2026-01-16\)](#)
 - [v1.6.0 \(2025-12-28\)](#)
 - [v1.5.3 \(2025-12-07\)](#)
 - [v1.5.0 \(2025-12-06\)](#)
 - [v1.4.1 \(2025-12-06\)](#)
 - [v1.4.0 \(2025-12-03\)](#)
 - [v1.3.0 \(2025-12-02\)](#)
 - [v1.2.0 \(2025-11-13\)](#)
 - [v1.1.0 \(2025-10-25\)](#)
 - [v1.0.0 \(2025-10-20\)](#)

NeoGeo中文几何描述 DSL 规范 v1.8.8

作者: Barry Chen

目录

- 前言
- DSL 设计原则
- 形式化语法定义 (EBNF)
- 基础元素 (Entities)
- 几何关系 (Relations)
- 操作型构造指令 (Operations)
- 图形结构 (Shapes)
- 宏指令与展开规则 (Macros)
- 圆与弧 (Circles & Arcs)
- 角度与长度表达
- 点的位置关系 (高级)
- 几何变换与折叠操作
- 坐标系与函数图像 (Coordinate & Functions)
- 模糊语义消歧指令 (Disambiguation)
- 中文表达对照表
- 合法性规则 (Validity Rules)
- 标准化与规范化 (Canonicalization)
- 标注示例
- 版本管理与变更日志

1. 前言

本 DSL (Domain-Specific Language) 用于将中文几何绘图指令转换为结构化、可解析的形式逻辑表示，用于：

- 中文 $\rightarrow$ DSL 模型训练 (NL $\rightarrow$ DSL)
- DSL $\rightarrow$ 图像生成 (DSL $\rightarrow$ Diagram)
- DSL $\rightarrow$ 几何求解 (Symbolic / Numeric Solver)
- 几何图像理解 (Diagram Parsing)
- 多轮几何编辑 (Geometric Editing Agent)

DSL 必须具备：

- 语义明确**：每条指令对应一个明确的几何概念、操作或者skills

- **可解析性**：可被solver自动解析、验证和执行
- **可扩展性**：支持通过宏指令扩展高层语义
- **与实际几何一致**：所有约束在几何上可满足
- **适合 LLM 生成**：格式简洁，便于语言模型学习和生成
- **贴合中文习惯**：支持中文几何题中常见的"操作型"、"隐式性"表达

本规范定义所有合法 token，任何人员和模型必须严格遵循，不可进行随意篡改和未经允许的调整。

2. DSL 设计原则

2.1 一条 DSL = 一个基本几何事实（Atomic Fact）

每条 DSL 指令表达一个原子几何事实，不能同时表达多个独立事实。

<code>parallel(AB, CD)</code>	✓ 表达一个平行关系
<code>angle(AOB)=40</code>	✓ 表达一个角度值
<code>on(C, circle_0)</code>	✓ 表达一个点在圆上

2.2 所有关键字必须小写

<code>circle(0)</code>	✓
<code>Circle(0)</code>	✗
<code>TRIANGLE</code>	✗

2.3 点名必须为大写字母（或带 prime 后缀）

合法点名格式：

<code>A, B, C, D</code>	# 单个大写字母
<code>A1, B2, C3</code>	# 大写字母 + 数字

O, P, Q	# 常用于圆心或特殊点
B_prime, C_prime	# 带 _prime 后缀（表示变换后的点）

2.4 线段用两个点名连写表示

segment(AB)	# 线段 AB
parallel(AB, CD)	# AB 平行于 CD
equal(AB, AC)	# AB 等于 AC

2.5 与中文描述结构对齐

DSL 设计尽量与中文题目的句子结构保持对应关系：

中文表达	DSL 对应
在△ABC中	<code>triangle(A, B, C)</code>
点 D 在 BC 上	<code>on(D, BC)</code>
连接 AD	<code>connect(A, D)</code>
AB = AC	<code>equal(AB, AC)</code>

2.6 简单核心 + 可扩展宏

- 核心层只保留基本图元和基本约束
- 高层概念（如中线、高、角平分线）通过宏指令定义，由解释器展开为基础约束

3. 形式化语法定义（EBNF）

以下使用扩展巴科斯-诺尔范式（EBNF）定义 DSL 的完整语法规则：

```
(* 程序结构 *)
program      ::= statement*
statement    ::= decl_stmt | constraint_stmt | op_stmt | macro_stmt | assign_stmt
```



```
| coord_stmt | function_stmt | function_transform | function_geom  
| disambig_stmt
```

(* 声明语句 *)

```
decl_stmt ::= point_decl | line_decl | segment_decl | ray_decl  
          | circle_decl | arc_decl | sector_decl | shape_decl
```

```
point_decl ::= "point" "(" POINT ")"  
           | "points" "(" POINT ("," POINT)* ")"
```

```
line_decl ::= "line" "(" SEGMENT ")"  
           | "line" "(" ID "," POINT "," POINT ")"
```

```
segment_decl ::= "segment" "(" SEGMENT ")"
```

```
ray_decl ::= "ray" "(" POINT "," POINT ")"
```

```
circle_decl ::= "circle" "(" POINT ")"  
             | "circle" "(" POINT "," r" "=" NUMBER ")"  
             | "circle" "(" POINT "," radius" "=" SEGMENT ")"
```

```
arc_decl ::= "arc" "(" POINT "," POINT "," CIRCLE_REF ")"
```

```
sector_decl ::= "sector" "(" POINT "," POINT "," POINT ")"  
              | "sector" "(" POINT "," POINT "," POINT "," angle" "=" NUMBER  
              ")"  
              | "sector" "(" POINT "," POINT "," POINT "," r" "=" NUMBER ")"
```

```
shape_decl ::= "triangle" "(" POINT "," POINT "," POINT ")"  
            | "quadrilateral" "(" POINT "," POINT "," POINT "," POINT ")"  
            | "polygon" "(" POINT ("," POINT)+ ")"  
            | "regular_polygon" "(" NUMBER "," POINT ("," POINT)+ ")"  
            | "rectangle" "(" POINT "," POINT "," POINT "," POINT ")"  
            | "parallelogram" "(" POINT "," POINT "," POINT "," POINT ")"  
            | "square" "(" POINT "," POINT "," POINT "," POINT ")"  
            | "trapezoid" "(" POINT "," POINT "," POINT "," POINT ")" (*
```

v1.8.3: 梯形 *)

(* 约束语句 *)

```
constraint_stmt ::= relation_constraint | position_constraint | measure_constraint
```

```
relation_constraint ::= "parallel" "(" LINE "," LINE ")"  
                    | "perpendicular" "(" LINE "," LINE ")"  
                    | "equal" "(" SEGMENT "," SEGMENT ")"  
                    | "equal_angle" "(" ANGLE "," ANGLE ")"  
                    | "collinear" "(" POINT "," POINT "," POINT ("," POINT)* ")"  
                    | "concylic" "(" POINT "," POINT "," POINT "," POINT ")"  
                    | "congruent" "(" TRIANGLE_ID "," TRIANGLE_ID ")"  
                    | "similar" "(" TRIANGLE_ID "," TRIANGLE_ID ")"  
                    | "tangent" "(" POINT "," CIRCLE_REF ")"  
                    | "tangent" "(" LINE "," CIRCLE_REF ")"  
                    | "bisector" "(" LINE "," ANGLE ")"  
                    | "inscribed" "(" TRIANGLE_ID "," CIRCLE_REF ")"  
                    | "circumscribed" "(" CIRCLE_REF "," TRIANGLE_ID ")"  
                    | "circumcircle" "(" POINT "," POINT "," POINT "," POINT ")"  
                    | "incenter" "(" POINT "," POINT "," POINT "," POINT ")"
```

(* v1.8.8: 内心 *)

```

| "centroid" "(" POINT "," POINT "," POINT "," POINT ")"
(* v1.8.8: 重心 *)
| "orthocenter" "(" POINT "," POINT "," POINT "," POINT ")"
(* v1.8.8: 垂心 *)

position_constraint ::= "on" "(" POINT "," LOCATION ")"
| "midpoint" "(" POINT "," SEGMENT ")"
| "arc_midpoint" "(" POINT "," SEGMENT "," CIRCLE_REF (","
"arc_type" "=" STRING)? ")"
| "foot" "(" POINT "," POINT "," FOOT_LINE ")"
| "intersection" "(" POINT "," LINE_REF "," LINE_REF ")" (*
v1.8.1: 支持延长线 *)
| "intersection" "(" POINT "," LINE_REF "," CIRCLE_REF (","
"index" "=" NUMBER)? ")"
| "intersection" "(" POINT "," CIRCLE_REF "," CIRCLE_REF (","
"index" "=" NUMBER)? ")"
| "trisection" "(" POINT "," SEGMENT "," "near" "=" POINT ")"
| "perpendicular_bisector" "(" POINT "," SEGMENT ")"
| "perpendicular_bisector" "(" SEGMENT "," SEGMENT ")" (*
v1.8.2: 线段是中垂线 *)

measure_constraint ::= "angle" "(" ANGLE ")" "=" NUMBER
| "angle" "(" ANGLE ")" "=" NUMBER "*" "angle" "(" ANGLE ")"
(* v1.6.7: angle ratio *)
| "length" "(" SEGMENT ")" "=" NUMBER
| "distance" "(" SEGMENT "," SEGMENT ")" "=" NUMBER (*
v1.8.5: 平行线段间距离 *)
| "ratio" "(" SEGMENT "," SEGMENT ")" "=" NUMBER
| "right_angle" "(" ANGLE ")"
| "sin" "(" ANGLE ")" "=" NUMBER (* v1.6.4: angle sine
value *)
| "cos" "(" ANGLE ")" "=" NUMBER (* v1.6.4: angle cosine
value *)
| "tan" "(" ANGLE ")" "=" NUMBER (* v1.6.4: angle tangent
value *)

(* 操作型构造语句 - 核心创新 *)
op_stmt ::= "connect" "(" POINT "," POINT ")"
| "extend" "(" SEGMENT "," POINT ("," "length" "=" NUMBER)? ")"
| "draw_perpendicular" "(" POINT "," LINE "," ID ")"
| "draw_parallel" "(" POINT "," LINE "," ID ")"
| "bisect_angle" "(" POINT "," POINT "," POINT "," ID ")"
| "draw_altitude" "(" POINT "," POINT "," POINT "," POINT ")"
| "draw_median" "(" POINT "," POINT "," POINT "," POINT ")"

(* 宏语句 *)
macro_stmt ::= "isosceles" "(" POINT "," POINT "," POINT ")"
| "isosceles_right" "(" POINT "," POINT "," POINT ")"
| "equilateral" "(" POINT "," POINT "," POINT ")"
| "right_triangle" "(" POINT "," POINT "," POINT "," "right_at"
"=" POINT ")"
| "median" "(" POINT "," POINT "," POINT "," POINT ")"
| "altitude" "(" POINT "," POINT "," POINT "," POINT ")"
| "angle_bisector" "(" POINT "," POINT "," POINT "," POINT ")"
| "rotate_to" "(" POINT "," "source" "=" POINT "," "center" "="
POINT ","
"target" "=" SEGMENT ("," "direction" "=" DIRECTION_VAL)? ")"

```

DIRECTION_VAL ::= "cw" | "ccw"

(* 赋值语句 *)

assign_stmt ::= "angle" "(" ANGLE ")" "=" "angle" "(" ANGLE ")"
| "length" "(" SEGMENT ")" "=" "length" "(" SEGMENT ")"

(* 坐标系与函数语句 *)

coord_stmt ::= "coordinate_system" "(" POINT ")"
| "axis_x" "(" POINT ")"
| "axis_y" "(" POINT ")"
| "point_coord" "(" POINT ", " NUMBER ", " NUMBER ")"

function_stmt ::= basic_function | power_function | exp_log_function
| trig_function | inverse_trig_function | function_expr

basic_function ::= "line_function" "(" ID ", " VALUE ", " VALUE ")"
| "parabola" "(" ID ", " VALUE ", " VALUE ", " VALUE ")"
| "hyperbola" "(" ID ", " VALUE ")"

power_function ::= "power_function" "(" ID ", " VALUE ")"

exp_log_function ::= "exp_function" "(" ID ", " (VALUE | "e") ")"
| "log_function" "(" ID ", " (VALUE | "e") ")"

trig_function ::= "sin_function" "(" ID ", " VALUE ", " VALUE ", " VALUE ")"
| "cos_function" "(" ID ", " VALUE ", " VALUE ", " VALUE ")"
| "tan_function" "(" ID ")"
| "cot_function" "(" ID ")"
| "sec_function" "(" ID ")"
| "csc_function" "(" ID ")"

inverse_trig_function ::= "arcsin_function" "(" ID ")"
| "arccos_function" "(" ID ")"
| "arctan_function" "(" ID ")"
| "arccot_function" "(" ID ")"

function_expr ::= "function_expr" "(" ID ", " STRING ")"

(* 函数变换语句 *)

function_transform ::= "translate_function" "(" ID ", " ID ", " NUMBER ", " NUMBER ")"
| "scale_function" "(" ID ", " ID ", " NUMBER ", " NUMBER ")"
| "reflect_function" "(" ID ", " ID ", " AXIS ")"

(* 函数与几何交互 *)

function_geom ::= "function_point" "(" POINT ", " ID ")"
| "function_intersection" "(" POINT ", " ID ", " ID ")"
| "axis_intersection" "(" POINT ", " ID ", " AXIS ")"
| "tangent_point" "(" POINT ", " ID ", " ID ")"
| "tangent_line" "(" ID ", " ID ", " POINT ")"
| "asymptote" "(" ID ", " ID ")"

(* 模糊语义消歧语句 - v1.4 核心创新 *)

disambig_stmt ::= direction_stmt | region_stmt | bisector_on_stmt | choose_stmt

direction_stmt ::= "direction" "(" POINT ", " SEGMENT ", " "towards" "=" POINT ")"
| "on_extended" "(" POINT ", " SEGMENT ", " "towards" "=" POINT ")"

```

region_stmt      ::= "inside" "(" POINT "," REGION_REF ")"
                  | "outside" "(" POINT "," REGION_REF ")"
                  | "sector" "(" POINT "," POINT "," POINT "," POINT ")" (*
sector(P, O, A, B) - P在以O为圆心、从OA到OB的扇形内 *)
                  | "half_plane" "(" POINT "," SEGMENT "," "side" "=" SIDE ")"
                  | "opposite_sides" "(" POINT "," POINT "," SEGMENT ("," "strict"
"=" "true")? ")"
                  | "same_sides" "(" POINT "," POINT "," SEGMENT ("," "strict" "="
"true")? ")"
                  | "shaded" "(" REGION_REF ")"

bisector_on_stmt ::= "on" "(" POINT "," "bisector_" ANGLE "," "near" "=" POINT ")"

choose_stmt      ::= "choose" "(" POINT "," "on" "=" LOCATION ")"
                  | "choose" "(" POINT "," "in" "=" REGION_REF ")"

(* 终结符定义 *)
POINT            ::= [A-Z] [0-9]* | [A-Z] [0-9]* "_prime"
SEGMENT          ::= POINT POINT
LINE             ::= SEGMENT | ID
ANGLE            ::= POINT POINT POINT (* 角标识, 如 ABC 表示  $\angle ABC$ ; 连写用于约束如
angle(ABC)=60, 逗号分隔用于函数参数如 bisect_angle(A,B,C,l) *)
TRIANGLE_ID      ::= POINT POINT POINT
CIRCLE_REF       ::= "circle_" POINT
RAY_REF          ::= "ray_" POINT POINT (* 射线引用, 如 ray_AB *)
BISECTOR_REF     ::= "bisector_" POINT POINT POINT (* 角平分线引用, 如
bisector_ABC *)
TRIANGLE_REF     ::= "triangle_" POINT POINT POINT
SECTOR_REF       ::= "sector_" POINT POINT POINT (* 扇形引用, 如
sector_OAB, 连写 *)
RECTANGLE_REF    ::= "rectangle_" POINT POINT POINT POINT (* 矩形引用, 如
rectangle_ABCD *)
PARALLELOGRAM_REF ::= "parallelogram_" POINT POINT POINT POINT (* 平行四边形引用 *)
SQUARE_REF       ::= "square_" POINT POINT POINT POINT (* 正方形引用 *)
QUADRILATERAL_REF ::= "quadrilateral_" POINT POINT POINT POINT (* 四边形引用 *)
TRAPEZOID_REF    ::= "trapezoid_" POINT POINT POINT POINT (* v1.8.3: 梯形引用
*)
POLYGON_REF      ::= "polygon_" POINT+ (* 多边形引用, 如
polygon_ABCDEF *)
REGION_REF       ::= TRIANGLE_REF | CIRCLE_REF | SECTOR_REF
                  | RECTANGLE_REF | PARALLELOGRAM_REF | SQUARE_REF
                  | QUADRILATERAL_REF | TRAPEZOID_REF | POLYGON_REF
LOCATION          ::= SEGMENT | SEGMENT "_extended" | RAY_REF | CIRCLE_REF | FUNC_REF
FOOT_LINE        ::= SEGMENT | SEGMENT "_extended" (* v1.7.1: foot约束支持延长线 *)
LINE_REF         ::= LINE | LINE "_extended" (* v1.8.1: intersection支持延长线
*)
FUNC_REF         ::= ID (* 函数引用 *)
ID               ::= [a-z] [a-z0-9]*
SIGNED_NUMBER    ::= "-"? [0-9]+ ("." [0-9]+)? (* 支持负数, v1.6 *)
VALUE            ::= SIGNED_NUMBER | SYMBOL (* 数值或符号变量, v1.6 *)
SYMBOL           ::= [a-z] [a-z0-9]* (* 符号变量: a, b, k1 等 *)
NUMBER           ::= SIGNED_NUMBER (* 向后兼容别名 *)
STRING           ::= "'" [^"]* "'" (* 函数表达式字符串 *)
AXIS              ::= "x" | "y" | "origin" (* 坐标轴或原点 *)
SIDE              ::= "left" | "right" (* 半平面侧向 *)

```

4. 基础元素 (Entities)

4.1 点的声明

```
point(A)           # 声明单个点 A
points(A, B, C, D) # 批量声明多个点（推荐用于多点场景）
```

4.2 线的声明

```
line(AB)           # 直线 AB（经过点 A 和 B 的直线）
line(l, A, B)       # 命名直线 l，经过点 A 和 B（便于后续引用）
segment(AB)         # 线段 AB
ray(A, B)           # 射线 AB（从 A 出发经过 B）
```

4.3 圆的声明

```
circle(O)           # 圆心为 O 的圆
circle(O, r=5)       # 圆心为 O，半径为 5
circle(O, radius=AB) # 圆心为 O，半径等于线段 AB 的长度
```

4.4 基本图形声明

```
triangle(A, B, C)    # △ABC
quadrilateral(A, B, C, D) # 四边形 ABCD（按顺序连接）
polygon(A, B, C, D, E) # 多边形（顶点按顺序）
regular_polygon(5, A, B, C, D, E) # 正五边形（n=5，顶点按顺序）
rectangle(A, B, C, D) # 矩形 ABCD
parallelogram(A, B, C, D) # 平行四边形 ABCD
square(A, B, C, D)    # 正方形 ABCD
```

注意：

- `triangle(A, B, C)` 隐式引入三条边 AB、BC、CA
- 四边形类图形的顶点必须按顺序（顺时针或逆时针）排列

5. 几何关系 (Relations)

5.1 平行与垂直

```
parallel(AB, CD)      # AB // CD
perpendicular(AB, CD)  # AB ⊥ CD
```

5.2 等长关系

```
equal(AB, AC)          # AB = AC
ratio(AB, AC)=2         # AB : AC = 2 : 1
```

5.3 角度关系

```
equal_angle(ABC, DEF)  # ∠ABC = ∠DEF
angle(ABC)=angle(DEF)   # 等价写法
```

5.4 共线与共圆

```
collinear(A, B, C)      # 三点共线
collinear(A, B, C, D)   # 四点共线（可扩展）
conyclic(A, B, C, D)    # 四点共圆
```

5.5 全等与相似

```
congruent(ABC, DEF)      #  $\triangle ABC \cong \triangle DEF$ 
similar(ABC, DEF)        #  $\triangle ABC \sim \triangle DEF$ 
```

5.6 点的位置关系

```
on(P, AB)                 # 点 P 在线段 AB 上
on(P, AB_extended)        # 点 P 在线段 AB 的延长线上
on(P, ray_AB)              # 点 P 在射线 AB 上
on(P, circle_0)            # 点 P 在圆 O 上
on(P, axis_x)              # 点 P 在 x 轴上 ( $P.y = 0$ )
on(P, axis_y)              # 点 P 在 y 轴上 ( $P.x = 0$ )
```

坐标轴约束说明：

- `on(P, axis_x)` 表示点 P 在 x 轴上，等价于约束 $P.y = 0$
- `on(P, axis_y)` 表示点 P 在 y 轴上，等价于约束 $P.x = 0$
- 通常与 `coordinate_system(0)` 配合使用（见第 13 节）
- 也可独立使用，此时以默认原点 (0, 0) 为参考

5.7 切线关系

```
tangent(A, circle_0)      # A 是圆 O 上的切点（隐含存在一条过 A 的切线）
tangent(l, circle_0)      # 直线 l 与圆 O 相切
```

语义说明：

- `tangent(A, circle_0)`：声明点 A 是圆 O 的一个切点。这意味着存在一条经过 A 的直线与圆 O 相切于 A 点。通常与 `on(A, circle_0)` 配合使用。
- `tangent(l, circle_0)`：声明直线 l 与圆 O 相切。切点位置由其他约束确定。

对应中文：

- 「过点 A 作圆 O 的切线」→ 需要组合多个约束
- 「直线 l 与圆 O 相切于点 A」→ `tangent(l, circle_0) + on(A, l) + on(A, circle_0)`

5.8 角平分线关系

```
bisector(l, ABC)          # 直线 l 是  $\angle ABC$  的平分线
```

蕴含约束: `bisector(l, ABC)` 隐含 `on(B, l)` (角平分线必过顶点 B)

6. 操作型构造指令 (Operations)

这是本项目的核心创新点，用于直接承接中文几何题中的"动词式操作"描述。

6.1 连接

```
connect(A, D)             # 连接 AD
```

语义展开: 隐式引入 `segment(AD)`

对应中文: 「连接 AD」「连接点 A 与点 D」

6.2 延长

```
extend(AB, D)             # 延长 AB 到点 D  
extend(AB, D, length=3)   # 延长 AB 到点 D, 使  $BD = 3$ 
```

语义展开: 生成 `on(D, AB_extended)` + 可选的长度约束

对应中文: 「延长 AB 至点 D」「将 AB 延长到 D」

6.3 作垂线

```
draw_perpendicular(P, AB, l)  # 过点 P 作直线 AB 的垂线, 命名为 l  
draw_perpendicular(P, l1, l2) # 过点 P 作直线 l1 的垂线 l2
```


语义展开：生成 `line(l, ...) + perpendicular(l, AB) + on(P, l)`

对应中文：「过点 P 作 AB 的垂线」「从 P 向 AB 作垂线」

6.4 作平行线

```
draw_parallel(P, AB, l)          # 过点 P 作直线 AB 的平行线，命名为 l
```

语义展开：生成 `line(l, ...) + parallel(l, AB) + on(P, l)`

对应中文：「过点 P 作 AB 的平行线」「经过 P 作一条直线平行于 AB」

6.5 作角平分线

```
bisect_angle(A, B, C, l)        # 作  $\angle ABC$  的平分线，命名为 l
```

语义展开：生成角平分线约束 `bisector(l, ABC)`

bisector 语义说明：

- `bisector(l, ABC)` 完整表达" l 是 $\angle ABC$ 的角平分线"
- 蕴含约束：`on(B, l)`（角平分线必过顶点 B）
- 几何性质： l 平分 $\angle ABC$ ，即 $\angle ABL = \angle LBC$ （ L 为 l 上任意点，仅作语义描述，不进入可执行展开）
- 如需显式使用角平分线上的点，应使用 `angle_bisector(P, A, B, C)` 宏

对应中文：「作 $\angle ABC$ 的平分线」「作角 B 的平分线」

6.6 作高

```
draw_altitude(A, B, C, H)      # 从顶点 A 向对边 BC 作高，垂足为 H
```

语义展开：生成 `foot(H, A, BC) + segment(AH) + perpendicular(AH, BC)`

对应中文：「从 A 向 BC 作高」「作 $\triangle ABC$ 中 BC 边上的高」

6.7 作中线

```
draw_median(A, B, C, M)          # 从顶点 A 向对边 BC 的中点 M 作中线
```

语义展开：生成 `midpoint(M, BC)` + `segment(AM)`

对应中文：「作 $\triangle ABC$ 的中线 AM 」「连接 A 与 BC 的中点」

7. 图形结构 (Shapes)

7.1 基本多边形

```
triangle(A, B, C)                # 三角形
quadrilateral(A, B, C, D)        # 四边形
polygon(A, B, C, D, E)           # 多边形（至少 3 个顶点）
regular_polygon(5, A, B, C, D, E) # 正五边形（n 与顶点数必须一致）
```

7.1.1 正多边形声明

```
regular_polygon(5, A, B, C, D, E) # 正五边形
regular_polygon(6, A, B, C, D, E, F) # 正六边形
```

语义：

- 第一个参数 `n` 是边数，后续按顺序给出 `n` 个顶点
- 隐含约束：所有边等长，所有内角相等
- `n` 必须是整数且 `n >= 3`
- 顶点数量必须与 `n` 完全一致

对应中文：

- 「正五边形 $ABCDE$ 」→ `regular_polygon(5, A, B, C, D, E)`
- 「正 n 边形」→ `regular_polygon(n, ...)`（标注时需替换为具体数字）

7.2 特殊三角形（宏指令）

7.2.1 等腰三角形

```
isosceles(A, B, C)      # 等腰三角形, AB = AC
```

隐含约束: `triangle(A, B, C) + equal(AB, AC)`

7.2.2 等腰直角三角形

```
isosceles_right(A, B, C)  # 等腰直角三角形, AB = AC,  $\angle BAC = 90^\circ$ 
```

隐含约束: `triangle(A, B, C) + equal(AB, AC) + right_angle(BAC)`

7.2.3 等边三角形

```
equilateral(A, B, C)     # 等边三角形
```

隐含约束: `triangle(A, B, C) + equal(AB, BC) + equal(BC, AC)`

7.2.4 直角三角形

```
right_triangle(A, B, C, right_at=B)  # 直角三角形, 直角在 B 处
```

隐含约束: `triangle(A, B, C) + perpendicular(AB, CB)`

7.3 特殊四边形

7.3.1 矩形

```
rectangle(A, B, C, D)     # 矩形 ABCD
```

隐含约束:

- `parallel(AB, CD) + parallel(BC, AD)`
- `perpendicular(AB, BC)`
- `equal(AB, CD) + equal(BC, AD)`

7.3.2 平行四边形

```
parallelogram(A, B, C, D)    # 平行四边形 ABCD
```

隐含约束：

- `parallel(AB, CD) + parallel(BC, AD)`
- `equal(AB, CD) + equal(BC, AD)`

7.3.3 梯形 (v1.8.3)

```
trapezoid(A, B, C, D)       # 梯形 ABCD
```

隐含约束：

- `parallel(AB, CD)` - AB 和 CD 为两条平行的底边
- BC 和 AD 为两条腰（不要求平行）

注意：与平行四边形的区别是只有一对对边平行，不添加 `parallel(BC, AD)` 约束。

对应中文：「梯形 ABCD，其中 AB // CD」

7.3.4 正方形

```
square(A, B, C, D)          # 正方形 ABCD
```

隐含约束： `rectangle(A, B, C, D) + equal(AB, BC)`

8. 宏指令与展开规则（Macros）

宏指令是为简化表达和提升可读性而设计的高阶构造，在解释执行时会自动展开为基础指令集合。

8.0 命名与参数设计原则

本 DSL 区分两类高阶指令，它们的参数顺序设计遵循不同的原则：

类型	前缀	参数顺序	语义风格
操作指令	<code>draw_*</code>	操作主体在前，结果在后	"从 A 向 BC 作高，垂足为 H"
宏指令	无前缀	结果点在前，依赖点在后	"H 是 A 到 BC 的垂足"

对比示例：

操作指令	宏指令	语义
<code>draw_altitude(A, B, C, H)</code>	<code>altitude(H, A, B, C)</code>	从 A 向 BC 作高，垂足为 H
<code>draw_median(A, B, C, M)</code>	<code>median(M, A, B, C)</code>	从 A 向 BC 中点作中线，中点为 M

两者语义等价，展开后的约束相同。选择哪种取决于原始中文表达的风格：

- 操作式表达（如"从 A 向 BC 作高"）→ 使用 `draw_*` 指令
- 声明式表达（如"H 是 A 到 BC 的垂足"）→ 使用宏指令

8.1 中线

```
median(M, A, B, C)
```

展开为：

```
midpoint(M, BC)
segment(AM)
```

语义：M 是 BC 的中点，AM 是从顶点 A 到对边中点的中线

8.2 高

```
altitude(H, A, B, C)
```

展开为：

```
foot(H, A, BC)
segment(AH)
perpendicular(AH, BC)
```

语义：H 是从 A 到 BC 的垂足，AH 是高

8.3 角平分线

```
angle_bisector(P, A, B, C)
```

展开为：

```
on(P, bisector_ABC)
equal_angle(ABP, PBC)
```

语义：P 在 $\angle ABC$ 的平分线上

8.4 垂直平分线约束

8.4.1 点在垂直平分线上

```
perpendicular_bisector(P, AB)
```

语义：点 P 在线段 AB 的垂直平分线上

约束条件： $|PA| = |PB|$ （P 到 A、B 的距离相等）

对应中文：「点 P 在 AB 的垂直平分线上」

8.4.2 线段是中垂线 (v1.8.2)

```
perpendicular_bisector(AB, CD)
```

语义：线段 AB 是 CD 的垂直平分线（AB 垂直平分 CD）

约束条件：

- 1. CD 的中点 M 在直线 AB 上（A、B、M 共线）
- 2. $AB \perp CD$ （向量点积为 0）

对应中文：「AB 是 CD 的中垂线」「AB 垂直平分 CD」

示例：

```
triangle(A, B, C)
perpendicular_bisector(DE, BC)  # DE 是 BC 的中垂线
```

8.5 特殊三角形宏展开表

宏指令	展开结果
isosceles(A, B, C)	triangle(A, B, C) + equal(AB, AC)
equilateral(A, B, C)	triangle(A, B, C) + equal(AB, BC) + equal(BC, CA)
isosceles_right(A, B, C)	triangle(A, B, C) + equal(AB, AC) + right_angle(BAC)
right_triangle(A, B, C, right_at=B)	triangle(A, B, C) + perpendicular(AB, CB)

8.6 旋转至目标 (rotate_to) (v1.8.4)

```
rotate_to(A1, source=A, center=C, target=BE)
rotate_to(A1, source=A, center=C, target=BE, direction=cw)
```

```
rotate_to(A1, source=A, center=C, target=BE, direction=ccw)
```

展开为：

```
equal(CA1, CA)
on(A1, BE)
```

语义：`rotate_to` 是宏指令，在 Extractor 阶段展开为基础约束。数学上等价于求"以 center 为圆心、|center-source| 为半径的圆与目标线段 target 的交点"。

与 `rotate` 的本质区别：

指令	性质	角度	Layer
<code>rotate(D, source=C, center=O, angle=60)</code>	确定性变换，2个方程完全确定D	必须显式指定	Layer 6
<code>rotate_to(A1, source=A, center=C, target=BE)</code>	宏展开为约束组合，A1 由求解器确定	隐式确定	Layer 5

与"连续旋转"的语义差异：

- `rotate_to` 的数学本质是圆 $\cap$ 线段的交点，不是"从 source 出发连续旋转扫过的第一个交点"
- 当圆与 target 有两个交点时，`direction` 参数用于初始化引导 solver 收敛到期望的交点
- `direction` 是软约束/初始化提示（与 `inside`、`half_plane` 等消歧指令一致），不是硬约束
- 在绝大多数几何题场景中，圆与 target 只有一个交点，不存在歧义

`direction` 参数说明：

- `direction=cw`：顺时针方向（从 source 到 result 的旋转方向为顺时针）
- `direction=ccw`：逆时针方向
- 省略时：不指定方向偏好，solver 取最近的有效交点

对应中文：

- 「将线段 CA 绕点 C 按顺时针方向旋转，点 A 落在边 BE 上记为 A1」→ `rotate_to(A1, source=A, center=C, target=BE, direction=cw)`

- 「以 C 为圆心、CA 为半径画弧，交 BE 于点 A1」→ `rotate_to(A1, source=A, center=C, target=BE)`

9. 圆与弧 (Circles & Arcs)

9.1 圆的声明

```
circle(O)           # 圆心为 O 的圆
circle(O, r=5)       # 指定半径
circle(O, radius=AB) # 半径等于线段 AB
```

9.2 点与圆的关系

```
on(A, circle_O)      # 点 A 在圆 O 上
tangent(A, circle_O) # 点 A 是圆 O 的切点
```

9.3 弧

```
arc(A, B, circle_O) # 圆 O 上从 A 到 B 的弧（声明）
on(E, arc_AB)       # 点 E 在弧 AB 上（需先声明 arc(A, B, circle_O)）
```

弧上点约束说明：

- `arc_AB` 引用格式表示从 A 到 B 的弧（逆时针方向的劣弧）
- 使用前必须先声明 `arc(A, B, circle_O)` 以关联圆心
- 约束等价于：点在圆上 + 角度在弧范围内

9.4 扇形声明 (v1.8.0)

```
sector(O, A, B)      # 以 O 为圆心，从 OA 到 OB 的扇形
sector(O, A, B, angle=120) # 指定圆心角 120°
```

```
sector(0, A, B, r=5)      # 指定半径 5
```

语义：声明一个扇形区域，用于渲染和约束。

隐含约束：

语法形式	生成的约束
<code>sector(0, A, B)</code>	<code>equal(OA, OB)</code>
<code>sector(0, A, B, angle=120)</code>	<code>equal(OA, OB) + angle(AOB)=120</code>
<code>sector(0, A, B, r=5)</code>	<code>length(OA)=5 + length(OB)=5</code>

对应中文：「以 O 为圆心、OA 和 OB 为边的扇形」「扇形 OAB」

与消歧语法的区别：

语法	参数数量	用途	示例
<code>sector(0, A, B, ...)</code>	3个点 + 可选参数	扇形声明（用于渲染）	<code>sector(0, A, B, angle=60)</code>
<code>sector(P, 0, A, B)</code>	4个点	消歧约束（P在扇形内）	<code>sector(P, 0, A, B)</code>

注意：

- 扇形声明 `sector(0, A, B)` 会自动引入 `|OA| = |OB|` 约束
- 扇形引用使用 `sector_OAB` 格式（如 `shaded(sector_OAB)`）
- 消歧语法 `sector(P, 0, A, B)` 见第 14.3.3 节

9.5 内接与外接

```
inscribed(ABC, circle_0)    # △ABC 内接于圆 0
circumscribed(circle_0, ABC) # 圆 0 外接于 △ABC
```

9.6 半圆

```
semicircle(O, A, B)          # 以O为圆心、AB为直径的半圆
```

语义: 声明一个以 O 为圆心、AB 为直径的半圆

隐含约束:

- `midpoint(O, AB)` - O 是 AB 的中点
- `circle(O, radius=OA)` - 创建以 O 为圆心、OA 为半径的圆
- `on(A, circle_0) + on(B, circle_0)` - A、B 在圆上

对应中文: 「以点 O 为圆心、AB 为直径的半圆」

使用场景: 当几何题目明确提到"半圆"且给出直径端点时使用

示例:

```
triangle(A, B, C)
semicircle(O, A, B)      # AB 为直径的半圆
on(D, circle_0)         # 点 D 在半圆上
```

注意:

- 半圆在约束求解时等同于完整的圆，区别仅在于语义表达
- 如需限制点只在半圆弧上（而非完整圆），需额外添加角度或位置约束

9.7 三角形四心 (v1.8.8)

```
circumcircle(O, A, B, C)    # O 是 △ABC 的外心（已有，v1.5.2）
incenter(I, A, B, C)        # I 是 △ABC 的内心
centroid(G, A, B, C)        # G 是 △ABC 的重心
orthocenter(H, A, B, C)     # H 是 △ABC 的垂心
```

语义:

关键字	几何定义	约束本质
<code>circumcircle(O, A, B, C)</code>	O 是三边垂直平分线的交点	$ OA = OB = OC $

关键字	几何定义	约束本质
<code>incenter(I, A, B, C)</code>	I 是三个内角角平分线的交点	$\text{dist}(I, AB) = \text{dist}(I, BC) = \text{dist}(I, CA)$
<code>centroid(G, A, B, C)</code>	G 是三条中线的交点	$G = (A + B + C) / 3$
<code>orthocenter(H, A, B, C)</code>	H 是三条高的交点	$HA \perp BC$ 且 $HB \perp AC$

对应中文：

- 「O 是 $\triangle ABC$ 的外心」 $\rightarrow$ `circumcircle(O, A, B, C)`
- 「I 是 $\triangle ABC$ 的内心」 $\rightarrow$ `incenter(I, A, B, C)`
- 「G 是 $\triangle ABC$ 的重心」 $\rightarrow$ `centroid(G, A, B, C)`
- 「H 是 $\triangle ABC$ 的垂心」 $\rightarrow$ `orthocenter(H, A, B, C)`
- 「O 是 $\triangle ABC$ 的外接圆圆心」 $\rightarrow$ `circumcircle(O, A, B, C)`
- 「I 是 $\triangle ABC$ 的内切圆圆心」 $\rightarrow$ `incenter(I, A, B, C)`

等价展开形式（理解用，实际不展开）：

- `incenter(I, A, B, C)  $\equiv$  angle_bisector  $\times$  3 + intersection`
- `centroid(G, A, B, C)  $\equiv$  midpoint  $\times$  3 + intersection`
- `orthocenter(H, A, B, C)  $\equiv$  foot  $\times$  3 + intersection`

拓扑排序：归入 Layer 4（位置约束 — 三角形中心子类别）

10. 角度与长度表达

10.1 角度

```
angle(ABC)=40           #  $\angle ABC = 40^\circ$ 
angle(ABC)=90           #  $\angle ABC = 90^\circ$ 
right_angle(ABC)        #  $\angle ABC$  是直角（等价于 angle(ABC)=90）
```

10.2 线段长度

```
length(AB)=5           # AB 的长度为 5
length(AB)=length(CD)  # AB 和 CD 等长（等价于 equal(AB, CD)）
```

10.2.1 数学表达式支持 (v1.6.9)

DSL 支持在数值参数位置使用数学表达式，Parser 阶段自动求值为小数。

支持的表达式类型

类型	示例	求值结果
分数	$1/2, -7/3, 5/4$	$0.5, -2.333..., 1.25$
根号	$\sqrt{3}, \text{sqrt}(5), 2\sqrt{5}$	$1.732..., 2.236..., 4.472...$
混合表达式	$(1+\sqrt{3})/2, 2\sqrt{5}/3$	$1.366..., 1.490...$
基础运算	$2+3, 5-1, 2*3$	$5, 4, 6$

使用示例

```
# 分数长度
length(AB)=1/2           # AB 长度为 0.5
length(CD)=-7/3          # CD 长度为 -2.333...

# 根号长度
length(AB)=sqrt(3)        # AB 长度为 1.732...
length(CD)=2*sqrt(5)      # CD 长度为 4.472...

# 混合表达式
length(AB)=(1+sqrt(3))/2  # AB 长度为 1.366...
point_coord(A, 2*sqrt(5)/3, 0) # A 坐标为 (1.490..., 0)

# 角度表达式
angle(ABC)=180/3          # ∠ABC = 60°
angle(DEF)=sqrt(3)*30     # ∠DEF ≈ 51.96°

# 比例表达式
ratio(AB, CD)=sqrt(2)     # AB:CD = sqrt(2):1 ≈ 1.414:1
```

适用范围

数学表达式可用于所有数值参数位置：

- 长度约束: `length(AB)=√3`
- 角度约束: `angle(ABC)=180/3`
- 坐标值: `point_coord(A, 1/2, √3)`
- 比例系数: `ratio(AB, CD)=√2`
- 圆半径: `circle(0, radius=2√5)` (当radius为数值时)
- 旋转角度: `rotate(D, source=C, center=0, angle=180/3)`

注意事项

- 1. 求值时机: 表达式在 Parser 阶段立即求值为 `float`, 不保留符号形式
- 2. 求值精度: 使用 SymPy 求值, 精度约 15 位小数
- 3. Unicode 支持: 同时支持 `√` (Unicode U+221A) 和 `sqrt()` 函数
- 4. 向后兼容: 原有纯数值 DSL (`3.14`, `0.5`) 完全不受影响
- 5. 错误处理: 无效表达式 (如 `1/0`) 会触发 `ParseError`
- 6. 运算符优先级: 遵循标准数学优先级 (括号 > 乘除 > 加减)

符号约定

符号	含义	示例
<code>+</code>	加法	<code>1+2 → 3</code>
<code>-</code>	减法/负号	<code>5-1 → 4</code> , <code>-3 → -3</code>
<code>*</code>	乘法	<code>2*3 → 6</code> , <code>2√5 → 4.472...</code>
<code>/</code>	除法	<code>1/2 → 0.5</code> , <code>2√5/3 → 1.490...</code>
<code>√</code>	平方根	<code>√3 → 1.732...</code>
<code>sqrt()</code>	平方根函数	<code>sqrt(5) → 2.236...</code>
<code>()</code>	分组	<code>(1+√3)/2 → 1.366...</code>

对应中文表达

中文	DSL (v1.6.9+, 推荐)	DSL (旧式, 仍支持)
AB 长为二分之一	<code>length(AB)=1/2</code>	<code>length(AB)=0.5</code>
AB 长为根号 3	<code>length(AB)=√3</code>	<code>length(AB)=1.732</code>
AB 长为 2 倍根号 5	<code>length(AB)=2√5</code>	<code>length(AB)=4.472</code>

中文	DSL (v1.6.9+, 推荐)	DSL (旧式, 仍支持)
点 B 横坐标为负三分之七	<code>point_coord(B, -7/3, y)</code>	<code>point_coord(B, -2.333, y)</code>
半径为根号 2	<code>circle(0, radius=√2)</code>	<code>circle(0, radius=1.414)</code>

实现细节

表达式求值流程：

```
DSL 输入: length(AB)=2√5/3
↓
Lexer: [LENGTH, LPAREN, IDENTIFIER(AB), RPAREN, EQUALS, EXPR("2√5/3")]
↓
Parser.parse_number():
  → evaluate_expression("2√5/3")
  → SymPy: "2√5/3" → "2*sqrt(5)/3" → 1.4907119849998598
↓
AST: LengthConstraint(seg=('A','B'), value=1.4907119849998598)
↓
Solver: 使用 float 值进行约束求解
```

与角度比例约束的区别：

语法	用途	求值时机	示例
<code>angle(AOB)=2*angle(BOC)</code>	角度比例关系	求解时动态计算	角度间存在倍数关系
<code>angle(AOB)=2*30</code>	数学表达式	解析时求值为 60.0	等价于 <code>angle(AOB)=60</code>

注意：2*angle(BOC) 中的 * 表示"角度比例"（运行时关系），而 2*30 中的 * 表示"算术乘法"（编译时常量）。

10.3 平行线段间距离 (v1.8.5)

```
distance(AB, CD)=2          # 平行线段 AB 与 CD 之间的垂直距离为 2
```

语义：线段 AB 所在直线与线段 CD 所在直线之间的垂直距离为 d。

前提条件：两线段已声明 `parallel(AB, CD)`，否则几何意义不明确。Extractor 会在缺少 `parallel` 声明时发出语义警告。

约束方程（双点约束）：对 AB 的两个端点 A、B 分别计算到直线 CD 的垂直距离，两者均应等于 d。这种设计在数值求解中间态（AB 和 CD 尚未严格平行时）提供更好的梯度引导。

对应中文：

中文表达	DSL
AB 与 CD 之间的距离为 2	<code>distance(AB, CD)=2</code>
两平行线之间的距离为 h	<code>distance(AB, CD)=h</code> (h 为具体数值)
梯形的高为 3	<code>distance(AB, CD)=3</code> (AB、CD 为两底)

备注：未来可扩展为 `distance(P, AB)=val` 表达点到直线距离，当前版本仅支持 `distance(segment, segment)=val` 形式。

10.4 比例关系

```
ratio(AB, CD)=2           # AB : CD = 2 : 1
ratio(AB, CD)=0.5         # AB : CD = 1 : 2
```

10.4.1 角度比例约束 (v1.6.7)

```
angle(AOB)=0.3333*angle(BOC)  # ∠AOB = (1/3) * ∠BOC
angle(ABC)=2*angle(DEF)       # ∠ABC = 2 * ∠DEF
angle(P1)=0.5*angle(P2)       # ∠P1 = 0.5 * ∠P2
```

语义：一个角度等于另一个角度的若干倍。

约束： $\angle AOB = k \times \angle BOC$ (k为比例系数)

对应中文：「 $\angle AOB=(1/3)\angle BOC$ 」「 $\angle ABC$ 是 $\angle DEF$ 的2倍」

注意：

- 比例系数k必须是数值字面量（不支持符号变量）

- v1.6.9+ 支持分数表达式：(1/3) 或 0.3333 均可
- 比例可以是任意正实数

10.4 角度单位约定 (v1.6)

本 DSL 中角度参数的单位约定如下：

上下文	单位	示例
angle(ABC)=N	度 (degree)	angle(ABC)=60 表示 60°
rotate(..., angle=N)	度 (degree)	angle=90 表示逆时针旋转 90°
sin_function(f, A, w, phi)	phi 为弧度 (radian)	phi=1.5708 ≈ π/2
cos_function(f, A, w, phi)	phi 为弧度 (radian)	同上

规则：几何约束中的角度统一使用度，三角函数相位参数使用弧度。

10.5 角度的三角函数值约束 (v1.6.4)

```
sin(ABC)=0.5           # ∠ABC 的正弦值为 0.5
cos(ABC)=0.866         # ∠ABC 的余弦值为 0.866 (≈ √3/2)
tan(ABC)=1             # ∠ABC 的正切值为 1 (即 45°)
```

语义：约束角度的三角函数值，而非角度本身。适用于题目直接给出三角函数值的情况。

参数范围：

- sin 和 cos：值必须在 [-1, 1] 范围内
- tan：值可以是任意实数（除接近 ±90° 时）

对应中文：

- 「tan∠ABC = 1/2」→ tan(ABC)=0.5
- 「cos∠ABC = √3/2」→ cos(ABC)=0.866
- 「sin∠ABC = 1/2」→ sin(ABC)=0.5

与 `angle(ABC)=N` 的区别：

语法	含义	示例
<code>angle(ABC)=60</code>	直接约束角度为 60°	$\angle ABC = 60^\circ$
<code>cos(ABC)=0.5</code>	约束角度使其余弦值为 0.5	$\cos \angle ABC = 0.5$ （即 60°）
<code>sin(ABC)=0.5</code>	约束角度使其正弦值为 0.5	$\sin \angle ABC = 0.5$ （即 30° 或 150°）
<code>tan(ABC)=1</code>	约束角度使其正切值为 1	$\tan \angle ABC = 1$ （即 45°）

11. 点的位置关系（高级）

11.1 中点

<code>midpoint(M, AB)</code>	# M 是 AB 的中点
------------------------------	--------------

11.2 三等分点

<code>trisection(C, AB, near=A)</code>	# C 是 AB 的三等分点，靠近 A
<code>trisection(D, AB, near=B)</code>	# D 是 AB 的三等分点，靠近 B

11.3 垂足

<code>foot(H, A, BC)</code>	# H 是从点 A 到直线 BC 的垂足
<code>foot(H, A, BC_extended)</code>	# H 是从点 A 到 BC 延长线的垂足 (v1.7.1)

语义说明：

- `foot(H, A, BC)`: BC 被解释为"过 B、C 两点的直线"（与第 16 节规则一致）
- `foot(H, A, BC_extended)`: 语义等价，但明确标注为延长线（提高可读性）
- 约束求解行为完全相同（line 始终视为无限直线）

对应中文：

- 「H 是 A 到 BC 的垂足」→ `foot(H, A, BC)`
- 「H 是 A 到 BC 延长线的垂足」→ `foot(H, A, BC_extended)`

11.4 交点

```
intersection(P, AB, CD)           # P 是直线 AB 与 CD 的交点
intersection(P, l1, l2)          # P 是直线 l1 与 l2 的交点
intersection(P, AB, circle_0)     # P 是直线 AB 与圆 O 的交点 (v1.6)
intersection(P, AB, circle_0, index=0) # 多交点时指定第一个交点
intersection(P, circle_01, circle_02) # P 是两圆的交点 (v1.6)
intersection(P, AB, CD_extended)   # P 是 AB 与 CD 延长线的交点 (v1.8.1)
intersection(P, AB_extended, CD)    # P 是 AB 延长线与 CD 的交点 (v1.8.1)
intersection(P, AB_extended, CD_extended) # 两条延长线的交点 (v1.8.1)
```

注意：

- 直线与圆、圆与圆可能有 0、1 或 2 个交点。使用 `index=0` 或 `index=1` 指定所需交点。
- `_extended` 后缀表示线段的延长线（即过两点的直线），语义上表明交点可能在线段本身之外。

11.5 弧中点 (v1.6.6)

```
arc_midpoint(D, BC, circle_0)           # D是圆O上劣弧BC的中点
arc_midpoint(D, BC, circle_0, arc_type='minor') # D是劣弧BC的中点（显式）
arc_midpoint(D, BC, circle_0, arc_type='major') # D是优弧BC的中点
```

语义：点D是圆O上连接B、C两点的弧的中点，即圆心角 $\angle BOD = \angle COD$ 。

约束说明：

- `arc_type='minor'`（默认）：劣弧（小于 180° 的弧）
- `arc_type='major'`：优弧（大于 180° 的弧）
- 等价约束：`on(D, circle_0) + equal_angle(BOD, COD) + 弧方向约束`

对应中文：「点D是劣弧BC的中点」「D平分弧BC」

11.6 延长线上的点

<code>on(E, AB_extended)</code>	# E 在 AB 的延长线上 (即 B 在 A、E 之间)
<code>on(F, BA_extended)</code>	# F 在 BA 的延长线上 (即 A 在 B、F 之间)

12. 几何变换与折叠操作

12.1 基本几何变换

<code>rotate(D, source=C, center=O, angle=60)</code>	# D 是 C 绕 O 逆时针旋转 60° 的结果
<code>translate(P, dx=2, dy=3)</code>	# 点 P 坐标沿x轴方向平移2, 沿y轴方向平移3
<code>scale(P, center=O, factor=2)</code>	# 点 P 以 O 为中心放大 2 倍
<code>reflect(D, C, AB)</code>	# D 是 C 关于直线 AB 的反射点 (v1.5.2)

rotate 语法说明:

- `rotate(D, source=C, center=O, angle=N)` - 唯一有效格式 - D 是 C 绕 O 逆时针旋转 N° 的结果
- 旋转角度为正时表示逆时针方向, 为负时表示顺时针方向
- 已移除: `rotate(P, center=O, angle=N)` 格式 (v1.5.3 起不再支持)

reflect 语法说明 (v1.5.2 新增):

- `reflect(P', P, AB)` - P' 是点 P 关于直线 AB 的反射点
- 反射满足: P 和 P' 到直线 AB 的距离相等, PP' 垂直于 AB
- 也可用于表示翻折操作后的点位置

12.2 折叠操作

折叠是中国几何题中常见的操作类型, 用于表示纸张折叠后的点位置变换:

<code>fold(P, P_prime, line)</code>	# 点 P 沿折痕 line 折叠后到达 P_prime 位置
-------------------------------------	---------------------------------

示例:

fold(C, C_prime, EF)

fold(B, B_prime, EF)

点 C 沿折痕 EF 折叠到 C'

点 B 沿折痕 EF 折叠到 B'

隐含约束：

- P 和 P_prime 关于折痕 line 对称
- `perpendicular(PP_prime, line)`
- P 到 line 的距离等于 P_prime 到 line 的距离

与 reflect 的关系：

`fold` 和 `reflect` 在几何上等价，但参数顺序不同：

指令	参数顺序	语义风格
<code>fold(源点, 目标点, 折痕)</code>	P, P', line	强调操作过程（从哪里折到哪里）
<code>reflect(目标点, 源点, 对称轴)</code>	P', P, line	强调结果声明（目标点是源点的像）

等价关系： $\text{fold}(C, C_prime, EF) \equiv \text{reflect}(C_prime, C, EF)$

推荐使用场景：

- 描述折纸、翻折操作时使用 `fold`
- 描述几何变换、对称关系时使用 `reflect`

13. 坐标系与函数图像（Coordinate & Functions）

本节定义坐标系和函数图像相关的 DSL 指令，用于支持解析几何类题目的标注。

13.1 坐标系声明

coordinate_system(0)

以 0 为原点的平面直角坐标系

说明：

- `coordinate_system(0)` 隐式声明原点 O 的坐标为 (0, 0)
- 隐式建立 x 轴和 y 轴（无需显式声明）
- 坐标轴上的点可通过 `point_coord` 或共线约束表示
- 原点 O 通常用大写字母 O 表示

坐标轴上的点约束：

若需约束点在坐标轴上，可使用 `on(P, axis_x)` 或 `on(P, axis_y)` 语法（见第 5.6 节）：

```
on(A, axis_x)           # 点 A 在 x 轴上 (A.y = 0)
on(B, axis_y)           # 点 B 在 y 轴上 (B.x = 0)
```

对应中文：

- 「点 A 在 x 轴上」→ `on(A, axis_x)`
- 「点 B 在 y 轴上」→ `on(B, axis_y)`
- 「直线 l 与 x 轴交于点 A」→ `on(A, axis_x) + on(A, l)` 或 `axis_intersection(A, l, x)`

辅助指令（通常不需要显式使用）：

```
axis_x(0)               # x 轴（经过原点 O 的水平轴）
axis_y(0)               # y 轴（经过原点 O 的垂直轴）
```

注意：`axis_x(0)` 和 `axis_y(0)` 为辅助指令，大多数情况下直接使用 `coordinate_system(0)` 即可，无需显式声明坐标轴。

13.2 坐标点声明

```
point_coord(A, 2, 3)    # 点 A 的坐标为 (2, 3)
point_coord(B, -1, 0)   # 点 B 的坐标为 (-1, 0)
point_coord(O, 0, 0)    # 原点 O 的坐标为 (0, 0)
```

对应中文：「点 A 的坐标为 (2, 3)」 「A(2, 3)」

13.3 基础函数图像声明

13.3.1 一次函数与二次函数

```
line_function(l, k, b)           # 直线函数  $y = kx + b$ , 命名为 l
line_function(l, -2, 8)          # 直线  $y = -2x + 8$ 

parabola(p, a, b, c)             # 抛物线  $y = ax^2 + bx + c$ , 命名为 p
parabola(p, 1, 0, 0)             # 抛物线  $y = x^2$ 
parabola(p, -1, 2, 3)           # 抛物线  $y = -x^2 + 2x + 3$ 
```

13.3.2 幂函数 (Power Functions)

```
power_function(f, n)             #  $y = x^n$ 
power_function(f, 2)             #  $y = x^2$  (等价于 parabola(f, 1, 0, 0))
power_function(f, 3)             #  $y = x^3$ 
power_function(f, 0.5)           #  $y = x^{(1/2)} = \sqrt{x}$ 
power_function(f, -1)            #  $y = x^{(-1)} = 1/x$  (等价于 hyperbola)
power_function(f, -2)            #  $y = x^{(-2)} = 1/x^2$ 
```

对应中文：

- 「幂函数 $y = x^3$ 」→ `power_function(f, 3)`
- 「根号函数 $y = \sqrt{x}$ 」→ `power_function(f, 0.5)`

13.3.3 反比例函数

```
hyperbola(h, k)                  # 反比例函数  $y = k/x$ , 命名为 h
hyperbola(h, 8)                  # 反比例函数  $y = 8/x$ 
hyperbola(h, -2)                 # 反比例函数  $y = -2/x$ 
```

13.3.4 指数函数 (Exponential Functions)

```
exp_function(f, a)               #  $y = a^x$  ( $a > 0, a \neq 1$ )
exp_function(f, e)               #  $y = e^x$  (自然指数,  $e \approx 2.718$ )
exp_function(f, 2)               #  $y = 2^x$ 
exp_function(f, 0.5)             #  $y = (1/2)^x = 0.5^x$ 
```

对应中文：

- 「指数函数 $y = e^x$ 」 → `exp_function(f, e)`
- 「指数函数 $y = 2^x$ 」 → `exp_function(f, 2)`

13.3.5 对数函数 (Logarithmic Functions)

```
log_function(f, a)          # y = log_a(x) (a > 0, a ≠ 1)
log_function(f, e)          # y = ln(x) (自然对数)
log_function(f, 10)         # y = lg(x) (常用对数)
log_function(f, 2)          # y = log2(x)
```

对应中文：

- 「对数函数 $y = \ln(x)$ 」 → `log_function(f, e)`
- 「对数函数 $y = \lg(x)$ 」 → `log_function(f, 10)`

13.3.6 三角函数 (Trigonometric Functions)

```
sin_function(f, A, w, phi)  # y = A·sin(ωx + φ)
sin_function(f, 1, 1, 0)    # y = sin(x)
sin_function(f, 2, 1, 0)    # y = 2sin(x)
sin_function(f, 1, 2, 0)    # y = sin(2x)
sin_function(f, 1, 1, 1.57) # y = sin(x + π/2)

cos_function(f, A, w, phi)  # y = A·cos(ωx + φ)
cos_function(f, 1, 1, 0)    # y = cos(x)

tan_function(f)              # y = tan(x)
cot_function(f)              # y = cot(x)
sec_function(f)              # y = sec(x) = 1/cos(x)
csc_function(f)              # y = csc(x) = 1/sin(x)
```

对应中文：

- 「正弦函数 $y = \sin(x)$ 」 → `sin_function(f, 1, 1, 0)`
- 「余弦函数 $y = \cos(x)$ 」 → `cos_function(f, 1, 1, 0)`
- 「正切函数 $y = \tan(x)$ 」 → `tan_function(f)`
- 「 $y = 2\sin(2x + \pi/3)$ 」 → `sin_function(f, 2, 2, 1.047)`

13.3.7 反三角函数 (Inverse Trigonometric Functions)

```
arcsin_function(f)          # y = arcsin(x), 定义域 [-1, 1]
arccos_function(f)          # y = arccos(x), 定义域 [-1, 1]
```



```
arctan_function(f)      # y = arctan(x)
arccot_function(f)     # y = arccot(x)
```

对应中文：

- 「反正弦函数 $y = \arcsin(x)$ 」 → `arcsin_function(f)`
- 「反正切函数 $y = \arctan(x)$ 」 → `arctan_function(f)`

13.4 通用函数表达式

用于表示复合函数和不常见的函数形式：

```
function_expr(f, "x*ln(x)")      # y = x·ln(x)
function_expr(f, "ln(x)/x")      # y = ln(x)/x
function_expr(f, "e^x + e^(-x)") # y = e^x + e^(-x) (双曲余弦相关)
function_expr(f, "x^2 * sin(x)") # y = x^2·sin(x)
function_expr(f, "sqrt(1-x^2)")  # y = √(1-x^2) (上半圆)
function_expr(f, "x^3 - 3*x")    # y = x^3 - 3x
```

表达式语法规则：

- 变量：`x`
- 运算符：`+`, `-`, `*`, `/`, `^` (幂运算)
- 内置函数：`sin`, `cos`, `tan`, `cot`, `sec`, `csc`, `arcsin`, `arccos`, `arctan`, `ln`, `log`, `exp`, `sqrt`, `abs`, `floor`, `ceil`
- 常数：`e` (自然常数), `pi` (圆周率), 数字

对应中文：

- 「 $y = x \cdot \ln(x)$ 」 → `function_expr(f, "x*ln(x)")`
- 「 $y = \ln(x)/x$ 」 → `function_expr(f, "ln(x)/x")`

13.5 函数变换

支持函数的平移、伸缩、对称变换：

```
translate_function(f, g, dx, dy) # g 是 f 沿 x 方向平移 dx, y 方向平移 dy
translate_function(f, g, 2, 0)   # g = f(x-2), 向右平移 2 个单位
translate_function(f, g, 0, 3)   # g = f(x) + 3, 向上平移 3 个单位
```

<code>scale_function(f, g, sx, sy)</code>	# g 是 f 沿 x 方向伸缩 sx 倍, y 方向伸缩 sy 倍
<code>scale_function(f, g, 2, 1)</code>	# $g = f(x/2)$, 横向拉伸 2 倍
<code>scale_function(f, g, 1, 2)</code>	# $g = 2f(x)$, 纵向拉伸 2 倍
<code>reflect_function(f, g, axis)</code>	# g 是 f 关于 $axis$ 的对称图像
<code>reflect_function(f, g, x)</code>	# $g = -f(x)$, 关于 x 轴对称
<code>reflect_function(f, g, y)</code>	# $g = f(-x)$, 关于 y 轴对称
<code>reflect_function(f, g, origin)</code>	# $g = -f(-x)$, 关于原点对称

对应中文:

- 「将 $f(x)$ 向右平移 2 个单位」→ `translate_function(f, g, 2, 0)`
- 「将 $f(x)$ 关于 y 轴对称」→ `reflect_function(f, g, y)`

13.6 函数与几何的交互

<code>function_point(P, f)</code>	# 点 P 在函数 f 的图像上
<code>function_intersection(P, f1, f2)</code>	# P 是函数 $f1$ 与 $f2$ 的交点
<code>axis_intersection(A, f, x)</code>	# A 是函数 f 与 x 轴的交点
<code>axis_intersection(B, f, y)</code>	# B 是函数 f 与 y 轴的交点
<code>tangent_point(C, l, f)</code>	# C 是直线 l 与曲线 f 的切点
<code>tangent_line(l, f, P)</code>	# l 是曲线 f 在点 P 处的切线
<code>asymptote(l, f)</code>	# l 是函数 f 的渐近线

对应中文:

- 「直线 l 与 x 轴交于点 A 」→ `axis_intersection(A, l, x)`
- 「直线 l 与反比例函数图像相切于点 C 」→ `tangent_point(C, l, h)`
- 「曲线 f 在点 P 处的切线」→ `tangent_line(l, f, P)`

参数顺序设计说明:

指令	参数顺序	设计原则
<code>tangent_point(C, l, f)</code>	点, 直线, 曲线	结果 (切点) 在前
<code>tangent_line(l, f, P)</code>	直线, 曲线, 点	结果 (切线) 在前, 参考位置 (切点) 在后

13.7 坐标系中的几何图形

在坐标系中，可以结合坐标点与几何图形：

```
coordinate_system(0)
point_coord(A, 0, 4)
point_coord(B, 4, 0)
point_coord(C, 1, 2)
triangle(0, A, B)
on(C, AB)
```

13.8 中文表达对照（坐标系与函数相关）

中文表达	DSL 转换
在平面直角坐标系中	coordinate_system(0)
点 A 的坐标为 (2, 3)	point_coord(A, 2, 3)
基础函数	
一次函数 $y = kx + b$	line_function(l, k, b)
二次函数 $y = ax^2 + bx + c$	parabola(p, a, b, c)
反比例函数 $y = k/x$	hyperbola(h, k)
幂函数	
幂函数 $y = x^3$	power_function(f, 3)
根号函数 $y = \sqrt{x}$	power_function(f, 0.5)
指数与对数函数	
指数函数 $y = e^x$	exp_function(f, e)
指数函数 $y = 2^x$	exp_function(f, 2)
对数函数 $y = \ln(x)$	log_function(f, e)
对数函数 $y = \lg(x)$	log_function(f, 10)
三角函数	
正弦函数 $y = \sin(x)$	sin_function(f, 1, 1, 0)
余弦函数 $y = \cos(x)$	cos_function(f, 1, 1, 0)
正切函数 $y = \tan(x)$	tan_function(f)

中文表达	DSL 转换
$y = 2\sin(2x + \pi/3)$	<code>sin_function(f, 2, 2, 1.047)</code>
反三角函数	
反正弦函数 $y = \arcsin(x)$	<code>arcsin_function(f)</code>
反余弦函数 $y = \arccos(x)$	<code>arccos_function(f)</code>
反正切函数 $y = \arctan(x)$	<code>arctan_function(f)</code>
复合函数	
$y = x \cdot \ln(x)$	<code>function_expr(f, "x*ln(x)")</code>
$y = \ln(x)/x$	<code>function_expr(f, "ln(x)/x")</code>
$y = e^x + e^{-x}$	<code>function_expr(f, "e^x + e^(-x)")</code>
函数变换	
将 $f(x)$ 向右平移 2 个单位	<code>translate_function(f, g, 2, 0)</code>
将 $f(x)$ 向上平移 3 个单位	<code>translate_function(f, g, 0, 3)</code>
将 $f(x)$ 关于 y 轴对称	<code>reflect_function(f, g, y)</code>
函数与几何交互	
与 x 轴交于点 A	<code>axis_intersection(A, f, x)</code>
与 y 轴交于点 B	<code>axis_intersection(B, f, y)</code>
图像经过点 P	<code>function_point(P, f)</code>
两函数图像相交于点 Q	<code>function_intersection(Q, f1, f2)</code>
曲线 f 在点 P 处的切线	<code>tangent_line(l, f, P)</code>

13.8.1 函数点坐标约束 (v1.7.0)

```
function_point_constrained(P, f, x=val) # 点 P 在函数 f 上, 且 P.x = val
function_point_constrained(P, f, y=val) # 点 P 在函数 f 上, 且 P.y = val
```

语义：点在函数图像上，且某一坐标（ x 或 y ）的值已知。

适用场景：

- 「点 P 在抛物线上，纵坐标为 -0.5」→ `function_point_constrained(P, p, y=-0.5)`
- 「点 Q 在双曲线上，横坐标为 2」→ `function_point_constrained(Q, h, x=2)`
- 「第三象限内抛物线上的点，纵坐标为 -3」→ `function_point_constrained(P, p, y=-3) + inside(P, half_plane_...)`

与 `function_point` 的区别：

约束类型	语法	语义	初始化
<code>function_point</code>	<code>function_point(P, f)</code>	P 在 f 上，位置完全未知	需要其他约束确定位置
<code>function_point_constrained</code>	<code>function_point_constrained(P, f, y=-0.5)</code>	P 在 f 上，y 坐标已知	自动求解 $f(x) = -0.5$

约束逻辑：

```
function_point_constrained(P, f, y=y0):
    1. 强约束: P.y = y0
    2. 强约束: P.y = f(P.x)
    3. 初始化: 求解  $f(x) = y0$ , 得到 P.x 的初始值

function_point_constrained(P, f, x=x0):
    1. 强约束: P.x = x0
```

2. 强约束: $P.y = f(P.x)$
3. 初始化: 直接计算 $P.y = f(x_0)$

注意事项:

1. 多解情况: 如果 $f(x) = y_0$ 有多个解 (如抛物线), Solver 会选择最接近 0 的解
 - 可通过额外约束 (如 `inside()`, $x > 0$ 等) 消除歧义
2. 无解情况: 如果 $f(x) = y_0$ 无解, 初始化会失败, 求解可能不收敛
 - 确保 y_0 在函数值域内
3. 数值表达式: 坐标值支持数学表达式 (v1.6.9+)

```
function_point_constrained(P, p, y=-1/2)      # 分数
function_point_constrained(Q, f, x=√3)        # 根号
function_point_constrained(R, h, y=2√5/3)     # 混合表达式
```

4. 向后兼容: 现有 `function_point(P, f)` 语法保持不变, 无需迁移

中文表达对照:

中文	DSL (v1.7.0+)
P 在抛物线上, 纵坐标为 -0.5	<code>function_point_constrained(P, p, y=-0.5)</code>
Q 在双曲线上, 横坐标为 2	<code>function_point_constrained(Q, h, x=2)</code>
第三象限内抛物线上一点 P, y=-3	<code>function_point_constrained(P, p, y=-3)</code>
P 在直线上, 过点 (2, f(2))	<code>function_point_constrained(P, l, x=2)</code>

14. 模糊语义消歧指令 (Disambiguation)

本节定义用于消除中文几何题中常见模糊表达的 DSL 指令。这是 v1.4 版本的核心创新, 旨在解决 LLM 从自然语言到 DSL 解析时遇到的歧义问题。

14.1 设计动机

中文几何题中存在大量模糊表达，例如：

- 「点 P 在 AB 的延长线上」→ 延长线方向不明确（向 A 还是向 B）
- 「在 $\triangle ABC$ 内取一点 P」→ 点的具体位置不明确
- 「在角平分线上取一点 Q」→ 靠近哪个端点不明确
- 「在 AB 上任取一点」→ 需要生成默认位置

这些模糊表达会导致：

1. LLM 无法可靠地进行语义解析
2. Solver 无法确定初始点位置
3. 渲染结果可能与题意不符

因此需要引入**消歧指令**，将模糊语义转化为明确的几何约束。

14.2 线段方向与位置指令

14.2.1 方向指示 **direction**

```
direction(P, AB, towards=A)    # P 位于 AB 上，且靠近 A 端
direction(P, AB, towards=B)    # P 位于 AB 上，且靠近 B 端
```

语义：用于消歧点在线段上的相对位置。

- **towards=A** 表示点 P 在 AB 的前半部分（距离 A 更近）
- **towards=B** 表示点 P 在 AB 的后半部分（距离 B 更近）

对应中文：

- 「P 在 AB 上靠近 A 的位置」→ **direction(P, AB, towards=A)**
- 「P 在 AB 上靠近端点 B」→ **direction(P, AB, towards=B)**

14.2.2 延长线方向 **on_extended**

```
on_extended(P, AB, towards=B)  # P 在 AB 延长线上（过 B 向外延伸）
on_extended(P, AB, towards=A)  # P 在 BA 延长线上（过 A 向外延伸）
```

语义：明确指定延长线的方向。

- `towards=B` 表示从 A 经过 B 继续延伸，P 在 B 的外侧
- `towards=A` 表示从 B 经过 A 继续延伸，P 在 A 的外侧

对应中文：

- 「延长 AB 至点 P」→ `on_extended(P, AB, towards=B)`
- 「延长 BA 至点 P」→ `on_extended(P, AB, towards=A)`
- 「P 在 AB 的延长线上」→ 需要根据上下文判断 `towards=A` 或 `towards=B`

与 `on(P, AB_extended)` 的区别：

- `on(P, AB_extended)` 不明确延长方向（旧版语法，保留兼容）
- `on_extended(P, AB, towards=B)` 明确指定延长方向（推荐使用）

14.2.3 射线声明 `ray`

```
ray(A, B)                # 从点 A 出发，经过点 B 的射线
```

语义：声明一条射线，起点为 A，方向由 B 确定。

对应中文：

- 「射线 AB」→ `ray(A, B)`
- 「从 A 出发经过 B 的射线」→ `ray(A, B)`

14.3 区域内外关系

14.3.1 内部 `inside`

```
inside(P, triangle_ABC)    # 点 P 在 △ABC 内部
inside(P, circle_O)        # 点 P 在 ⊙O 内部
inside(P, sector_AOB)      # 点 P 在扇形 AOB 内部
inside(P, rectangle_ABCD)  # 点 P 在矩形 ABCD 内部 (v1.6.3)
inside(P, parallelogram_ABCD) # 点 P 在平行四边形 ABCD 内部 (v1.6.3)
inside(P, square_ABCD)     # 点 P 在正方形 ABCD 内部 (v1.6.3)
inside(P, quadrilateral_ABCD) # 点 P 在四边形 ABCD 内部 (v1.6.3)
inside(P, polygon_ABCDEF)  # 点 P 在多边形 ABCDEF 内部 (v1.6.3)
```

语义：声明点在某个区域的内部。Extractor 会据此推断初始点坐标。

对应中文：

- 「在 $\triangle ABC$ 内取一点 P」→ `inside(P, triangle_ABC)`
- 「点 P 在圆 O 内」→ `inside(P, circle_O)`
- 「点 P 在矩形 ABCD 内」→ `inside(P, rectangle_ABCD)`
- 「点 P 在四边形 ABCD 内」→ `inside(P, quadrilateral_ABCD)`

Solver 处理：

- 不作为硬约束
- 用于推断初始点位置（如三角形重心附近）
- 可选：添加 soft constraint 验证

14.3.2 外部 `outside`

```
outside(P, triangle_ABC)      # 点 P 在  $\triangle ABC$  外部
outside(P, circle_O)         # 点 P 在  $\odot O$  外部
outside(P, rectangle_ABCD)    # 点 P 在矩形 ABCD 外部 (v1.6.3)
outside(P, parallelogram_ABCD) # 点 P 在平行四边形 ABCD 外部 (v1.6.3)
outside(P, square_ABCD)       # 点 P 在正方形 ABCD 外部 (v1.6.3)
outside(P, quadrilateral_ABCD) # 点 P 在四边形 ABCD 外部 (v1.6.3)
outside(P, polygon_ABCDEF)    # 点 P 在多边形 ABCDEF 外部 (v1.6.3)
```

语义：声明点在某个区域的外部。

对应中文：

- 「点 P 在 $\triangle ABC$ 外」→ `outside(P, triangle_ABC)`
- 「圆外一点 P」→ `outside(P, circle_O)`
- 「点 P 在矩形 ABCD 外」→ `outside(P, rectangle_ABCD)`
- 「点 P 在四边形 ABCD 外」→ `outside(P, quadrilateral_ABCD)`

14.3.3 扇形区域

扇形区域约束使用 `sector(P, O, A, B)` 定义，表示点 P 在以 O 为圆心、从 OA 到 OB 的扇形区域内。

```
sector(P, O, A, B)           # 点 P 在扇形 OAB 内部
```

语义：声明点在扇形区域内。角度按逆时针方向从 OA 到 OB 计算。

对应中文：

- 「点 P 在扇形 AOB 内」→ `sector(P, 0, A, B)`

14.3.4 半平面 `half_plane`

```
half_plane(P, AB, side=left)    # 点 P 在直线 AB 的左侧半平面
half_plane(P, AB, side=right)  # 点 P 在直线 AB 的右侧半平面
```

语义：声明点在直线划分的半平面内。

- `left`：沿 A→B 方向看，点在左侧
- `right`：沿 A→B 方向看，点在右侧

对应中文：

- 「P 在直线 AB 的一侧」→ 需要根据上下文判断 `side=left` 或 `side=right`
- 「P 与 C 在直线 AB 的同侧」→ 需要先确定 C 的位置

14.3.5 异侧/同侧约束 `opposite_sides` / `same_sides` (v1.8.6)

```
opposite_sides(A, D, PQ)        # A、D 在直线 PQ 异侧（软约束）
opposite_sides(A, D, PQ, strict=true) # A、D 在直线 PQ 异侧（硬约束）
same_sides(A, D, PQ)            # A、D 在直线 PQ 同侧（软约束）
same_sides(A, D, PQ, strict=true) # A、D 在直线 PQ 同侧（硬约束）
```

语义：声明两个点相对于一条直线的位置关系。

- 语义基于直线 `line(P, Q)`，而非有限线段 `segment(PQ)`
- 判定依据为叉积符号：`cross = (point - P) x (Q - P)`
- `opposite_sides`：两点的叉积符号相反 (`cross_A * cross_D < 0`)
- `same_sides`：两点的叉积符号相同 (`cross_A * cross_D > 0`)
- 点在直线上 (`|cross| < EPS`) 不属于任一侧，两种约束均视为不满足

软约束与硬约束：

- 默认为**软约束**（Layer 9 消歧），仅影响初始化坐标选择，与 `half_plane/inside/direction` 一致
- 加 `strict=true` 升级为**硬约束**，通过 `compute_error()` 惩罚违反，直接参与数值求解
- 硬约束 `to_equations()` 返回空（不等式无法表示为方程），仅通过数值优化器执行

退化保护：当 $P == Q$ 时直线退化，约束无意义，返回不满足/大惩罚值。

对应中文：

- 「点 A、D 在 PQ 异侧」→ `opposite_sides(A, D, PQ)`
- 「A、D 分居 BC 两侧」→ `opposite_sides(A, D, BC)`
- 「使点 A、D 在 PQ 异侧」→ `opposite_sides(A, D, PQ)` 或 `opposite_sides(A, D, PQ, strict=true)`
- 「A、D 在 BC 同侧」→ `same_sides(A, D, BC)`

与 `half_plane` 的区别：

特性	<code>half_plane</code>	<code>opposite_sides</code> / <code>same_sides</code>
参数	1 个点 + 方向	2 个点，无需指定方向
语义	绝对位置 (left/right)	相对位置关系
约束类型	仅软约束	软/硬可选

14.3.6 阴影区域标记 `shaded`

```
shaded(triangle_ABC)      # 标记  $\triangle ABC$  为阴影区域
shaded(sector_OAB)        # 标记扇形 OAB 为阴影区域
shaded(circle_O)          # 标记  $\odot O$  为阴影区域
```

语义：标记某个区域为阴影/填充区域，通常用于表示题目中需要强调或计算面积的部分。

对应中文：

- 「图中阴影部分」→ `shaded(region_ref)`
- 「求阴影部分的面积」→ 通过 `shaded` 标记目标区域

Solver/Renderer 处理：

- 渲染时对该区域进行填充显示
- 可用于面积计算的目标区域标识

14.4 角平分线消歧 `on(P, bisector, near=...)`

<code>on(P, bisector_ABC, near=B)</code>	# P 在 $\angle ABC$ 的平分线上, 靠近顶点 B
<code>on(P, bisector_ABC, near=C)</code>	# P 在 $\angle ABC$ 的平分线上, 远离顶点 B, 靠近边 BC 方向

语义：当在角平分线上取点时，消歧点的位置。

参数说明：

- `near=B` (顶点)：点靠近角的顶点
- `near=A` 或 `near=C` (边端点)：点远离顶点，靠近该边方向

对应中文：

- 「在 $\angle ABC$ 的平分线上取一点 P, P 靠近 B」 $\rightarrow$ `on(P, bisector_ABC, near=B)`
- 「角平分线与 BC 边交于点 D」 $\rightarrow$ `on(D, bisector_ABC, near=C) + on(D, BC)`

14.5 任选点 `choose`

14.5.1 线上任选点

<code>choose(P, on=AB)</code>	# 在线段 AB 上任选一点 P
<code>choose(P, on=circle_O)</code>	# 在圆 O 上任选一点 P
<code>choose(P, on=ray_AB)</code>	# 在射线 AB 上任选一点 P

语义：在指定位置随机/默认生成一个点。

Solver 处理：

- 在线段上：默认取中点或黄金分割点
- 在圆上：默认取特定角度位置（如 45° ）
- 在射线上：默认取距起点一定距离的点

对应中文：

- 「在 AB 上任取一点 P」 $\rightarrow$ `choose(P, on=AB)`
- 「在 $\odot O$ 上取一点 Q」 $\rightarrow$ `choose(Q, on=circle_O)`

14.5.2 区域内任选点

```
choose(P, in=triangle_ABC)      # 在 △ABC 内任选一点 P
choose(P, in=circle_O)         # 在 ⊙O 内任选一点 P
choose(P, in=sector_AOB)       # 在扇形 AOB 内任选一点 P
```

语义：在指定区域内随机/默认生成一个点。

Solver 处理：

- 在三角形内：默认取重心
- 在圆内：默认取圆心附近
- 在扇形内：默认取中心角对应的位置

对应中文：

- 「在 $\triangle ABC$ 内任取一点 P 」 $\rightarrow$ `choose(P, in=triangle_ABC)`

14.6 消歧指令的 Extractor 处理

ConstraintExtractor 对消歧指令的处理策略：

指令类型	处理方式
<code>direction</code>	添加 soft constraint，影响初始值生成
<code>on_extended</code>	生成 <code>on(P, segment_extended)</code> + 方向约束
<code>inside/outside</code>	推断初始坐标，不添加硬约束
<code>sector</code>	推断初始坐标，可选添加角度范围约束
<code>half_plane</code>	推断初始坐标，可选添加位置验证
<code>opposite_sides</code>	默认 soft constraint； <code>strict=true</code> 时添加硬约束
<code>same_sides</code>	默认 soft constraint； <code>strict=true</code> 时添加硬约束
<code>bisector_on</code> with <code>near</code>	生成角平分线约束 + 位置提示
<code>choose</code>	生成默认坐标，标记为可自由移动

14.7 中文表达对照（消歧相关）

中文表达	DSL 转换
延长 AB 至点 P	<code>on_extended(P, AB, towards=B)</code>
延长 BA 至点 P	<code>on_extended(P, AB, towards=A)</code>
P 在 AB 上靠近 A	<code>on(P, AB) + direction(P, AB, towards=A)</code>
P 在 $\triangle ABC$ 内部	<code>inside(P, triangle_ABC)</code>
P 在 $\odot O$ 外部	<code>outside(P, circle_O)</code>
P 在扇形 AOB 内	<code>inside(P, sector_OAB)</code>
P 在矩形 ABCD 内	<code>inside(P, rectangle_ABCD)</code>
P 在矩形 ABCD 外	<code>outside(P, rectangle_ABCD)</code>
P 在四边形 ABCD 内	<code>inside(P, quadrilateral_ABCD)</code>
P 在四边形 ABCD 外	<code>outside(P, quadrilateral_ABCD)</code>
P 在多边形内	<code>inside(P, polygon_ABCDEF)</code>
P 在多边形外	<code>outside(P, polygon_ABCDEF)</code>
P 在直线 AB 左侧	<code>half_plane(P, AB, side=left)</code>
A、D 分居 PQ 两侧	<code>opposite_sides(A, D, PQ)</code>
A、D 在 PQ 异侧	<code>opposite_sides(A, D, PQ)</code>
A、D 在 PQ 同侧	<code>same_sides(A, D, PQ)</code>
在 AB 上任取一点 P	<code>choose(P, on=AB)</code>
在 $\triangle ABC$ 内取一点 P	<code>choose(P, in=triangle_ABC)</code>
P 在角平分线上靠近 B	<code>on(P, bisector_ABC, near=B)</code>

15. 中文表达对照表

本节提供系统的中文几何表达与 DSL 的映射关系，用于指导标注和模型训练。

15.1 基础图形描述

中文表达	DSL 转换
在 $\triangle ABC$ 中 / 如图, $\triangle ABC$	<code>triangle(A, B, C)</code>
四边形 ABCD	<code>quadrilateral(A, B, C, D)</code>
正五边形 ABCDE	<code>regular_polygon(5, A, B, C, D, E)</code>
矩形 ABCD	<code>rectangle(A, B, C, D)</code>
平行四边形 ABCD	<code>parallelogram(A, B, C, D)</code>
正方形 ABCD	<code>square(A, B, C, D)</code>
$\odot O$ / 圆 O	<code>circle(O)</code>

15.2 点与位置关系

中文表达	DSL 转换
点 D 在 BC 上	<code>on(D, BC)</code>
点 E 在 AB 的延长线上	<code>on(E, AB_extended)</code>
点 P 在圆 O 上	<code>on(P, circle_O)</code>
M 是 AB 的中点	<code>midpoint(M, AB)</code>
H 是从 A 到 BC 的垂足	<code>foot(H, A, BC)</code>
P 是 AB 与 CD 的交点	<code>intersection(P, AB, CD)</code>
A、B、C 三点共线	<code>collinear(A, B, C)</code>
A、B、C、D 四点共圆	<code>concyclic(A, B, C, D)</code>

15.3 线段与角度关系

中文表达	DSL 转换
$AB = AC$	<code>equal(AB, AC)</code>
$AB \parallel CD$ / AB 平行于 CD	<code>parallel(AB, CD)</code>
$AB \perp CD$ / AB 垂直于 CD	<code>perpendicular(AB, CD)</code>
$\angle ABC = 40^\circ$	<code>angle(ABC)=40</code>

中文表达	DSL 转换
$\angle ABC = \angle DEF$	<code>equal_angle(ABC, DEF)</code>
$\angle ABC$ 是直角	<code>right_angle(ABC)</code>
AB 的长度为 5	<code>length(AB)=5</code>
AB 与 CD 之间的距离为 2	<code>distance(AB, CD)=2</code>
AB : CD = 2 : 1	<code>ratio(AB, CD)=2</code>

15.4 操作型表达

中文表达	DSL 转换
连接 AD	<code>connect(A, D)</code>
延长 AB 至点 D	<code>extend(AB, D)</code>
过点 P 作 BC 的垂线	<code>draw_perpendicular(P, BC, l)</code>
过点 P 作 BC 的平行线	<code>draw_parallel(P, BC, l)</code>
作 $\angle ABC$ 的平分线	<code>bisect_angle(A, B, C, l)</code>
从 A 向 BC 作高	<code>draw_altitude(A, B, C, H)</code>
作 BC 边上的中线	<code>draw_median(A, B, C, M)</code>
将 A 绕 C 旋转至落在 BE 上, 记为 A1	<code>rotate_to(A1, source=A, center=C, target=BE)</code>
将 CA 绕 C 顺时针旋转, A 落在 BE 上记为 A1	<code>rotate_to(A1, source=A, center=C, target=BE, direction=cw)</code>

15.5 特殊三角形

中文表达	DSL 转换
等腰三角形 ABC, $AB = AC$	<code>isosceles(A, B, C)</code>
等边三角形 ABC	<code>equilateral(A, B, C)</code>
等腰直角三角形, $AB=AC$, $\angle A=90^\circ$	<code>isosceles_right(A, B, C)</code>
直角三角形, $\angle B = 90^\circ$	<code>right_triangle(A, B, C, right_at=B)</code>

中文表达	DSL 转换
$\triangle ABC \cong \triangle DEF$	<code>congruent(ABC, DEF)</code>
$\triangle ABC \sim \triangle DEF$	<code>similar(ABC, DEF)</code>

15.6 可忽略的表达

以下中文表达通常作为题目引导语，不需要转换为 DSL：

中文表达	处理方式
如图所示 / 如图	忽略（图形描述起始标记）
已知 / 设	忽略（条件引导词）
求证 / 证明	忽略（问题类型标记）
求 / 计算	忽略（问题类型标记）

16. 合法性规则（Validity Rules）

所有 DSL 必须满足以下规则：

规则编号	规则描述	示例
V1	Token 必须出现在本规范定义的列表中	<code>circle(0)</code> ✓, <code>oval(0)</code> X
V2	点名必须是大写字母或大写字母+数字或带 <code>_prime</code> 后缀	<code>A</code> , <code>B1</code> , <code>C_prime</code> ✓
V3	所有关键字必须小写	<code>triangle</code> ✓, <code>Triangle</code> X
V4	几何度量角度（内角）必须在 (0, 180] 范围内	<code>angle(ABC)=40</code> ✓, <code>angle(ABC)=-10</code> X, <code>angle(ABC)=200</code> X
V4a	变换操作的角度允许负值（单位：度），负值表示顺时针	<code>rotate(..., angle=-30)</code> ✓ 表示顺时针旋转 30°

规则编号	规则描述	示例
V5	多边形顶点数必须 ≥ 3	<code>polygon(A, B, C)</code> ✓, <code>polygon(A, B)</code> X
V5a	正多边形边数必须为整数且 ≥ 3 , 且与顶点数量一致	<code>regular_polygon(5, A, B, C, D, E)</code> ✓, <code>regular_polygon(5.5, ...)</code> X
V6	圆的引用必须使用 <code>circle_0</code> 格式	<code>on(A, circle_0)</code> ✓, <code>on(A, 0)</code> X
V7	每个 DSL 函数的参数数量必须匹配定义	<code>triangle(A, B, C)</code> ✓, <code>triangle(A, B)</code> X
V8	线段标识必须由两个点名组成	<code>segment(AB)</code> ✓, <code>segment(A)</code> X
V9	引用的几何对象必须先声明 (推荐)	先 <code>triangle(A, B, C)</code> 再 <code>equal(AB, AC)</code>
V10	一条 DSL 只能表达一个原子事实	不能合并多个独立约束
V11	<code>tangent(A, circle_0)</code> 隐含 <code>on(A, circle_0)</code>	切点必须在圆上

V11 实现行为：

- **Validator (strict 模式)：**若存在 `tangent(A, circle_0)` 但缺少 `on(A, circle_0)`，报 error
- **Validator (autocorrect 模式)：**自动补全 `on(A, circle_0)` 并记录 warning
- **Canonicalization：**归一化时自动补全 `on(A, circle_0)`，确保评估一致性

角度语义说明：

- `angle(ABC)=N` 表示 $\angle ABC$ 的**内角**度数， $N \in (0, 180]$
- `rotate(..., angle=N)` 的 N 为旋转角度（单位：度），正值逆时针，负值顺时针
- 本 DSL 默认使用**内角**表示，不支持外角或有向角（如需扩展，另行定义）

类型检查规则

参数位置	期望类型	说明
<code>equal(X, Y)</code> 的 X, Y	SEGMENT	必须是线段标识符

参数位置	期望类型	说明
<code>angle(X)</code> 的 X	ANGLE	必须是三个点组成的角
<code>on(P, L)</code> 的 P	POINT	必须是点
<code>on(P, L)</code> 的 L	LOCATION	可以是线段、延长线、射线或圆
<code>parallel(L1, L2)</code> 的 L1, L2	LINE	必须是线段或命名直线

线段与直线的语义区分

上下文	SEGMENT 的语义	示例
<code>on(P, AB)</code>	线段 AB（P 在 A、B 之间，含端点）	点 P 在线段 AB 上
<code>on(P, AB_extended)</code>	线段 AB 的 延长线 （已在 EBNF LOCATION 中定义）	点 P 在 AB 延长线上
<code>collinear(A, P, B)</code>	三点 共线 （P 在直线 AB 上，不限于线段内）	点 P 在直线 AB 上
<code>parallel(AB, CD)</code>	过 A、B 两点的 直线	直线 AB 平行于直线 CD
<code>perpendicular(AB, CD)</code>	过 A、B 两点的 直线	直线 AB 垂直于直线 CD
<code>bisector(l, ABC)</code>	l 是 直线	直线 l 是 $\angle ABC$ 的平分线
<code>tangent(l, circle_0)</code>	l 是 直线	直线 l 与圆 O 相切

规则总结：

- 线段语义：**`on(P, AB)` 表示 P 在线段 AB 上（含端点，即 A、P、B 共线且 P 在 A、B 之间或与端点重合）
- 直线语义：**在直线关系约束（`parallel`, `perpendicular`, `bisector`, `tangent`）中，SEGMENT 一律解释为"过两点的直线"
- 直线上的点：**若需表达"P 在直线 AB 上（不限于线段）"，应使用 `collinear(A, P, B)` 或 `collinear(P, A, B)`

关于 AB_extended 的说明：

- `AB_extended` 已在 EBNF 的 `LOCATION` 终结符中定义：`LOCATION ::= SEGMENT | SEGMENT "_extended" | RAY_REF | ...`
- 因此 `on(P, AB_extended)` 是合法语法，表示"P 在线段 AB 的延长线上"
- 使用 `on_extended(P, AB, towards=B)` 可以更精确地指定延长方向（见第 14.2.2 节）

17. 标准化与规范化（Canonicalization）（v1.6）

本节定义 DSL 程序的标准化规则，用于训练数据一致性和评估指标准确性。

17.1 DSL 语句拓扑排序

DSL 语句必须按照**定义-引用顺序**排列，即对象必须先声明再使用。推荐的 9 层优先级如下：

层 级	语句类 型	示例
1	点声 明、坐 标系	<code>point(A), coordinate_system(O)</code>
2	图形声 明	<code>circle(O), triangle(A, B, C), regular_polygon(5, A, B, C, D, E), line_function(1, 2, 1)</code>
3	线段/ 射线/ 直线	<code>segment(AB), ray(A, B), line(1, A, B)</code>
4	位置约 束	<code>on(D, BC), midpoint(M, AB), foot(H, A, BC), intersection(...), incenter(I, A, B, C), centroid(G, A, B, C), orthocenter(H, A, B, C), circumcircle(O, A, B, C)</code>
5	构造操 作/宏	<code>extend(AB, D), draw_perpendicular(...), rotate_to(...)</code>
6	变换操 作	<code>rotate(...), reflect(...), fold(...)</code>

层 级	语句类 型	示例
7	几何关系	<code>equal(AB, CD), parallel(...), perpendicular(...), inscribed(...), circumscribed(...), similar(...), congruent(...), tangent(...)</code>
8	度量约束	<code>angle(ABC)=60, length(AB)=5, ratio(...)=2, distance(AB, CD)=2</code>
9	消歧/辅助	<code>inside(...), shaded(...), direction(...), opposite_sides(...), same_sides(...)</code>

17.1.1 Layer 2（图形声明） vs Layer 7（几何关系）的区别

这是一个容易混淆的关键点，需要特别注意：

Layer 2 - 图形声明：声明几何对象的**存在**

- `circle(O)` - 声明存在一个圆心为O的圆
- `triangle(A, B, C)` - 声明存在一个三角形ABC
- `regular_polygon(5, A, B, C, D, E)` - 声明存在一个正多边形并隐含等边等角约束
- `rectangle(A, B, C, D)` - 声明存在一个矩形ABCD
- `line_function(1, 2, 1)` - 声明存在一个函数线 $y=2x+1$

Layer 7 - 几何关系：描述已声明对象之间的**关系**

- `inscribed(ABC, circle_O)` - 三角形ABC **内接于** 圆O
- `circumscribed(ABC, circle_O)` - 三角形ABC **外接于** 圆O
- `similar(ABC, DEF)` - 三角形ABC **相似于** 三角形DEF
- `congruent(ABC, DEF)` - 三角形ABC **全等于** 三角形DEF
- `tangent(A, circle_O)` - 点A处 **切于** 圆O

关键原则：

1. **声明在前，关系在后**：必须先声明对象（Layer 2），才能描述对象间关系（Layer 7）
2. **依赖性检查**：`inscribed(ABC, circle_O)` 依赖于 `triangle(A,B,C)` 和 `circle(O)` 都已声明
3. **语义清晰度**：

-  正确: `triangle(A,B,C) → circle(0) → ... → inscribed(ABC, circle_0)`
-  错误: `inscribed(ABC, circle_0) → triangle(A,B,C)` (违反依赖顺序)

常见错误示例:

```
# 错误: inscribed 在 triangle 之前
inscribed(ABC, circle_0) # Layer 7
triangle(A, B, C)        # Layer 2 - 但ABC还未定义!

# 正确: 先声明, 后关系
circle(0)                # Layer 2
triangle(A, B, C)         # Layer 2
on(A, circle_0)           # Layer 4
on(B, circle_0)           # Layer 4
on(C, circle_0)           # Layer 4
inscribed(ABC, circle_0)  # Layer 7 - ABC和0都已定义
```

macros 的特殊性:

某些 macro 语句虽然看起来像声明, 但实际上会展开为声明+关系的组合:

- `isosceles(A, B, C) → triangle(A,B,C) (Layer 2) + equal(AB, AC) (Layer 7)`
- `circumcircle(0, A, B, C) → circle(0) (Layer 2) + on(A,circle_0) (Layer 4) + on(B,circle_0) (Layer 4) + on(C,circle_0) (Layer 4)`
- `incenter(I, A, B, C) → 等价于三条 angle_bisector + intersection, 但作为直接约束实现 (Layer 4)`
- `centroid(G, A, B, C) → 等价于三条 midpoint + intersection, 但作为直接约束实现 (Layer 4)`
- `orthocenter(H, A, B, C) → 等价于三条 foot + intersection, 但作为直接约束实现 (Layer 4)`

这些 macro 可以放在 Layer 2, 因为它们的主要作用是声明图形, 附带的约束会自动展开到正确的层级。三角形四心 (circumcircle, incenter, centroid, orthocenter) 归入 Layer 4 作为位置约束。

17.2 Operations 与 Macros 的展开

Operations (如 `connect`, `extend`) 和 Macros (如 `isosceles`) 是语法糖, 可以无损展开为基础语句。

原始形式	展开形式
<code>connect(A, D)</code>	<code>segment(AD)</code>
<code>isosceles(A, B, C)</code>	<code>triangle(A, B, C) + equal(AB, AC)</code>
<code>draw_altitude(A, B, C, H)</code>	<code>foot(H, A, BC) + segment(AH) + perpendicular(AH, BC)</code>
<code>fold(C, C', EF)</code>	<code>reflect(C', C, EF)</code>
<code>rotate_to(A1, source=A, center=C, target=BE)</code>	<code>equal(CA1, CA) + on(A1, BE)</code>

训练建议：评估时可以先将预测结果和标准答案都展开为 canonical form，再计算匹配度。

17.2.1 等价写法归一化

以下等价写法在评估时应归一化为 canonical form：

输入形式（兼容）	Canonical Form	说明
<code>angle(ABC)=angle(DEF)</code>	<code>equal_angle(ABC, DEF)</code>	角相等（无向内角）
<code>length(AB)=length(CD)</code>	<code>equal(AB, CD)</code>	长度相等
<code>connect(A, D)</code>	<code>segment(AD)</code>	连接两点
<code>fold(C, C', EF)</code>	<code>reflect(C', C, EF)</code>	折叠等价于反射
<code>tangent(A, circle_0)</code> 缺少 <code>on(A, circle_0)</code>	自动补全 <code>on(A, circle_0)</code>	V11 隐含约束

equal_angle 语义定义：

- `equal_angle(ABC, DEF)` 表示 $\angle ABC$ 与 $\angle DEF$ 的**无向内角**相等
- 与 V4 规则一致，均使用内角语义
- 不涉及有向角或角的方向性

归一化原则：模型输出和标准答案在评估前都应先归一化为 canonical form，再计算匹配度。

17.3 点名标准化

对于自动生成的辅助点，建议使用以下命名规则：

- 中点：M, M1, M2...
- 垂足：H, H1, H2...
- 交点：P, Q, P1...
- 变换后的点：A_prime, B_prime...

18. 标注示例

以下是完整的标注示例，展示从中文描述到 DSL 的转换过程。

18.1 示例 1：圆周角定理

中文描述：点 A、B、C 是⊙O 上的点， $\angle AOB=70^\circ$, $\angle ACB=35^\circ$

DSL 标注：

```
circle(0)
on(A, circle_0)
on(B, circle_0)
on(C, circle_0)
segment(OA)
segment(OB)
segment(AB)
segment(CA)
segment(CB)
angle(AOB)=70
angle(ACB)=35
```

完整 JSON 格式：

```
{
  "id": "00001",
  "text_cn": "点 A、B、C 是⊙O 上的点， $\angle AOB=70^\circ$ ,  $\angle ACB=35^\circ$ ",
  "dsl": [
    "circle(0)",
    "on(A, circle_0)",
    "on(B, circle_0)",
    "on(C, circle_0)",
```



```

    "segment(OA)",
    "segment(OB)",
    "segment(AB)",
    "segment(CA)",
    "segment(CB)",
    "angle(AOB)=70",
    "angle(ACB)=35"
  ],
  "meta": {
    "source": "圆周角定理基础题",
    "difficulty": "easy",
    "tags": ["circle", "inscribed_angle", "central_angle"]
  }
}

```

18.2 示例 2：矩形折叠问题

中文描述：折叠矩形ABCD，点C位于AD的中点C'，AB=9, BC=6

DSL 标注：

```

rectangle(A, B, C, D)
point(E)
point(F)
point(C_prime)
point(B_prime)
midpoint(C_prime, AD)
on(E, BC)
on(F, CD)
segment(EF)
segment(EC_prime)
segment(FC_prime)
segment(EB_prime)
segment(FB_prime)
fold(C, C_prime, EF)
fold(B, B_prime, EF)
length(AB)=9
length(BC)=6

```

18.3 示例 3：平行四边形与垂足

中文描述：平行四边形ABDC中，P是BC上一点，Q是BC延长线上一点，H是从D到BP的垂足

DSL 标注：

```
parallelogram(A, B, D, C)
point(P)
point(Q)
point(H)
on(P, BC)
on(Q, BC_extended)
segment(AD)
segment(AB)
segment(BD)
segment(DC)
segment(DH)
segment(DP)
segment(DQ)
foot(H, D, BP)
perpendicular(DH, BC)
```

18.4 示例 4：等腰三角形与中线

中文描述：在 $\triangle ABC$ 中， $AB=AC$ ， M 是 BC 的中点，连接 AM

DSL 标注：

```
isosceles(A, B, C)
midpoint(M, BC)
connect(A, M)
```

或等价的展开形式：

```
triangle(A, B, C)
equal(AB, AC)
midpoint(M, BC)
segment(AM)
```

18.5 示例 5：作图操作

中文描述：在 $\triangle ABC$ 中，过点 A 作 BC 的垂线，垂足为 H ，作 $\angle BAC$ 的平分线交 BC 于点 D

DSL 标注：

```
triangle(A, B, C)
draw_altitude(A, B, C, H)
```

```
bisect_angle(B, A, C, l)
intersection(D, l, BC)
on(D, BC)
```

19. 版本管理与变更日志

19.1 版本号规则

本规范采用语义化版本号（Semantic Versioning）：

- 主版本号（MAJOR）：不兼容的 API 变更
- 次版本号（MINOR）：向后兼容的功能性新增
- 修订号（PATCH）：向后兼容的问题修正

19.2 向后兼容性承诺

- 新增指令：旧数据集仍然有效，新指令为可选
- 废弃指令：保留解析支持至少一个主版本周期，验证器给出 warning
- 语义重大改动：提供迁移脚本 `migrate_dsl_vX_to_vY.py`

19.3 变更日志

v1.8.8 (2026-02-28)

三角形四心 DSL 支持 (新增)：

- 新增 `incenter(I, A, B, C)` 语法：I 是 $\triangle ABC$ 的内心（三角内角平分线交点）
- 新增 `centroid(G, A, B, C)` 语法：G 是 $\triangle ABC$ 的重心（三条中线交点）
- 新增 `orthocenter(H, A, B, C)` 语法：H 是 $\triangle ABC$ 的垂心（三条高的交点）
- 三者与已有 `circumcircle(O, A, B, C)` 共同构成完整的三角形四心体系
- 引入 `TriangleCenterConstraint` 基类，统一四心约束架构
- 约束公式设计兼顾符号可解性与数值稳定性（`to_equations` 与 `compute_error` 分离）
- 拓扑排序归入 Layer 4（位置约束 — 三角形中心子类别）
- 新增 `INCENTER`、`CENTROID`、`ORTHOCENTER` token

- 新增 `IncenterDecl`、`CentroidDecl`、`OrthocenterDecl` AST 节点
- 新增 `IncenterConstraint`、`CentroidConstraint`、`OrthocenterConstraint` 约束类

v1.8.7 (2026-03-03)

`regular_polygon` 规范补齐（文档与实现统一）：

- 在 EBNF `shape_decl` 中补充 `regular_polygon(NUMBER, POINT...)` 产生式
- 新增第 7.1.1 节，明确 `regular_polygon` 的语义与参数约束
 - `n` 必须为整数且 ≥ 3
 - 顶点数量必须与 `n` 一致
 - 隐含等边与等角约束
- 在第 15 章中文对照表补充“正五边形”映射
- 在第 16 章合法性规则补充 V5a（正多边形专项规则）
- 在第 17 章拓扑排序与 Layer 2 示例中明确 `regular_polygon` 属于图形声明

v1.8.6 (2026-03-03)

`opposite_sides` / `same_sides` 异侧同侧约束 (新增)：

- 新增 `opposite_sides(A, D, PQ)` 语法：声明两点在直线 PQ 异侧
- 新增 `same_sides(A, D, PQ)` 语法：声明两点在直线 PQ 同侧
- 默认为软约束（Layer 9 消歧），可选 `strict=true` 升级为硬约束
- 语义基于直线 `line(P, Q)`，非有限线段
- 点在直线上视为不属于任一侧（EPS 边界策略）
- 退化直线保护（`P == Q` 时返回大惩罚值）
- 硬约束通过 `compute_error()` 惩罚实现，`to_equations()` 返回空
- 主动初始化优化：extractor 检测到不满足时镜像翻转点坐标
- 新增 `OPPOSITE_SIDES`、`SAME_SIDES`、`STRICT` token
- 新增 `OppositeSidesConstraint`、`SameSidesConstraint` AST 节点
- 新增软约束类 `OppositeSidesSoftConstraint`、`SameSidesSoftConstraint`
- 新增硬约束类 `OppositeSidesHardConstraint`、`SameSidesHardConstraint`

v1.8.5 (2026-03-02)

`distance` 平行线段间距离约束 (新增)：

- 新增 `distance(AB, CD)=d` 语法，用于表达两条平行线段之间的垂直距离
- 双点约束设计：同时约束 AB 的两个端点到直线 CD 的距离，增强数值求解稳定性

- 退化保护：线段退化为点时返回大惩罚值
- Extractor 语义检查：缺少 `parallel` 声明时发出 `warnings.warn()`
- 拓扑排序归入 Layer 8（度量约束）
- 新增 `DISTANCE` token
- 新增 `DistanceConstraint` AST 节点
- 新增 `ParallelDistanceConstraint` 约束类
- 扩展性预留：parser 结构支持未来 `distance(P, AB)=val` 形式

v1.8.4 (2026-02-28)

`rotate_to` 旋转至目标宏指令 (新增):

- 新增 `rotate_to(A1, source=A, center=C, target=BE)` 宏指令：表达"将某点绕中心旋转至落在指定线段上"
 - 宏指令（非几何变换），在 Extractor 阶段 `desugar` 为基础约束
 - 展开为 `EqualLengthConstraint`（距离保持）+ `PointOnSegmentConstraint`（目标位置）
 - 数学语义：圆（圆心=`center`，半径= $|\text{center-source}|$ ） $\cap$ 线段（`target`）的交点
 - 可选 `direction=cw|ccw` 参数：软约束/初始化提示，不是硬约束
 - 拓扑排序归入 Layer 5（构造/宏操作）
 - 新增 EBNF `macro_stmt` 产生式
 - 新增 `ROTATE_TO`、`TARGET` token
 - 新增 `RotateToConstraint` AST 节点
 - 对应中文：「将线段 CA 绕 C 旋转，A 落在 BE 上记为 A1」

v1.8.3 (2026-01-28)

`trapezoid` 梯形语法 (新增):

- 新增 `trapezoid(A, B, C, D)` 语法：声明一个梯形，其中 `AB // CD` 为两条平行的底边
 - 隐含约束：`parallel(AB, CD)`
 - BC 和 AD 为两条腰，不添加平行约束
 - 与平行四边形的区别：只有一对对边平行
 - 支持区域引用：`trapezoid_ABCD`
 - 支持 `inside(P, trapezoid_ABCD)` 和 `outside(P, trapezoid_ABCD)` 约束

v1.8.2 (2026-01-28)

perpendicular_bisector 线段语法 (新增):

- 新增线段-线段语法: `perpendicular_bisector(AB, CD)` 表示线段 AB 是 CD 的垂直平分线
 - 约束 1: CD 的中点在直线 AB 上
 - 约束 2: $AB \perp CD$
 - 与现有 `perpendicular_bisector(P, AB)` (点在中垂线上) 语法兼容共存
 - 通过第一个参数长度区分: 单字符为点版本, 多字符为线段版本

v1.8.1 (2026-01-28)

intersection 延长线语法 (新增):

- 新增 `_extended` 后缀支持: `intersection` 语句支持 `AB_extended` 语法表示线段延长线
 - `intersection(P, AB, CD_extended)` - P 是 AB 与 CD 延长线的交点
 - `intersection(P, AB_extended, CD)` - P 是 AB 延长线与 CD 的交点
 - `intersection(P, AB_extended, CD_extended)` - 两条延长线的交点
 - 新增 EBNF 终结符 `LINE_REF ::= LINE | LINE "_extended"`
 - 新增 `IntersectionConstraint.obj1_extended` 和 `obj2_extended` 字段
 - 向后兼容: 现有 `intersection(P, AB, CD)` 语法不变

v1.8.0 (2026-01-28)

扇形声明语法 (新增):

- 新增 `sector(0, A, B, ...)` 声明语法: 声明扇形区域, 用于渲染和约束
 - `sector(0, A, B)` - 基本形式, 隐含 `equal(OA, OB)`
 - `sector(0, A, B, angle=120)` - 指定圆心角, 隐含角度约束
 - `sector(0, A, B, r=5)` - 指定半径, 隐含长度约束
 - 新增 EBNF 产生式 `sector_decl`
 - 新增 `SectorDecl` AST 节点
 - 新增 `visit_sector_decl` 约束提取方法

与消歧语法的区别:

语法	参数	用途
<code>sector(0, A, B, ...)</code>	3点+可选	扇形声明
<code>sector(P, 0, A, B)</code>	4点	消歧 (P在扇形内)

实现状态:

- Lexer: 复用现有 `SECTOR` token
- Parser: 根据参数数量区分声明和消歧
- AST: 新增 `SectorDecl` 节点
- Extractor: 新增 `visit_sector_decl` 方法
- GeometryModel: 新增 `SectorInfo` 数据类
- 渲染: SVG/TikZ 支持扇形渲染

v1.7.1 (2026-01-25)

foot 约束语法增强:

- **支持延长线语法:** `foot(H, A, BC_extended)`
 - 新增 EBNF 终结符 `FOOT_LINE ::= SEGMENT | SEGMENT "_extended"`
 - 新增 `FootConstraint.is_extended` 字段 (用于 AST 往返)
 - 与 `on()` 约束保持一致的行为
 - 向后兼容: `foot(H, A, BC)` 仍然有效

语义澄清:

- `foot(H, A, BC)`: BC 被解释为"过 B、C 两点的直线" (与规范第 16 节一致)
- `foot(H, A, BC_extended)`: 语义等价, 但明确标注为延长线 (提高可读性)
- 约束求解行为完全相同 (line 始终视为无限直线)

实现状态:

- Parser: 增强 `parse_foot_constraint()` 支持 `_extended` 后缀
- AST: `FootConstraint` 新增 `is_extended: bool` 字段
- Extractor: 文档化 `_extended` 处理逻辑
- 测试用例: 完整覆盖语法解析和约束求解

v1.7.0 (2026-01-24)

核心功能增强:

- **函数点坐标约束:** 新增 `function_point_constrained` 约束类型
 - 语法: `function_point_constrained(P, f, x=val)` 或 `function_point_constrained(P, f, y=val)`
 - 语义: 点 P 在函数 f 上, 且某一坐标 (x 或 y) 值已知
 - 新增 `FUNCTION_POINT_CONSTRAINED` token 类型

- 新增 `FunctionPointConstrainedConstraint` AST 节点和约束类
- Parser 支持命名参数语法: `x=val` 或 `y=val`
- Extractor 智能初始化: 自动求解 $f(x) = y_0$ 或直接计算 $y = f(x_0)$

解决的问题:

- 之前无法表达"函数上某坐标已知的点":
 - `point_coord(P, m, -0.5)` ← 错误: `m` 是符号变量, 无法解析
 - `function_point(P, p)` ← 不完整: 需要额外约束确定 `P` 的位置
- 新语法直接表达: `function_point_constrained(P, p, y=-0.5)`

适用场景:

- 「点 `P` 在抛物线上, 纵坐标为 `-0.5`」
- 「第三象限内函数图像上的点, `y` 坐标为 `-3`」
- 「双曲线上横坐标为 `2` 的点」

向后兼容性:

- 保留 `function_point(P, f)` 语法, 无需迁移现有数据集
- 新约束作为补充, 用于更精确的坐标控制

实现状态:

- 新增 `solver/parser/lexer.py` `FUNCTION_POINT_CONSTRAINED` token
- 新增 `solver/parser/ast_nodes.py` `FunctionPointConstrainedConstraint` 节点
- 新增 `solver/parser/parser.py`
`parse_function_point_constrained_constraint` 方法
- 新增 `solver/constraints/relations.py`
`FunctionPointConstrainedConstraint` 约束类
- 新增 `solver/constraints/extractor.py`
`visit_function_point_constrained_constraint` 方法
- 新增 `solver/constraints/extractor.py` `_solve_function_for_x` 辅助方法
- 文档完整 (第 13.8.1 节)
- 测试用例覆盖 (`test_function_point_constrained.py`)

v1.6.9 (2026-01-24)

核心功能增强:

- **数学表达式支持:** 在所有数值参数位置支持数学表达式

- 新增 `TokenType.EXPR` token 类型
- 新增 `solver/parser/expr_eval.py` 表达式求值模块（基于 SymPy）
- Lexer 增强：`read_expression()` 方法识别分数、根号、混合表达式
- Parser 增强：`parse_number()` 支持 EXPR token 并自动求值
- 支持分数：`1/2`, `-7/3`, `5/4`
- 支持根号：`√3`, `sqrt(5)`, `2√5`（Unicode 和 ASCII 两种写法）
- 支持混合表达式：`(1+√3)/2`, `2√5/3`
- 支持基础运算：`+`, `-`, `*`, `/`, 括号优先级

适用范围：

- 长度约束：`length(AB)=√3`
- 角度约束：`angle(ABC)=180/3`
- 坐标值：`point_coord(A, 1/2, √3)`
- 比例系数：`ratio(AB, CD)=√2`
- 圆半径：`circle(0, radius=2√5)`（数值半径）
- 旋转角度：`rotate(..., angle=180/3)`

求值策略：

- 表达式在 **Parser 阶段立即求值为 float**
- 不保留符号形式（`√3` → `1.7320508...`）
- 使用 SymPy 求值，精度约 15 位小数
- 向后兼容：纯数值 DSL（`3.14`）行为不变

实现状态：

- 新增 `solver/parser/expr_eval.py`（172 行）
- 修改 `solver/parser/lexer.py`（新增 EXPR token，`read_expression` 方法）
- 修改 `solver/parser/parser.py`（增强 `parse_number` 方法）
- 文档完整（第 10.2.1 节）
- 测试用例覆盖（`test_expr_eval.py`）

注意：v1.6.7 的角度比例约束说明已更新，明确"分数需预先计算为小数"的限制已被解除。

v1.6.8 (2026-01-24)

文档补充：

- **半圆语法文档补充：**在第 9.5 节添加 `semicircle(0, A, B)` 语法说明

- `semicircle` 语法已在 solver 代码中完整实现，但 DSL 规范文档缺失
- 语义：声明以 O 为圆心、AB 为直径的半圆
- 隐含约束展开：O 是 AB 中点 + 创建圆 O（半径= $|OA|$ ）+ A、B 在圆上
- 补充使用场景、示例代码和注意事项

实现状态：

- Lexer/Parser/AST/Extractor 均已实现（v1.5.x 之前）
- 测试用例完整覆盖（`test_parser_complete.py`, `test_extractor_complete.py`）
- 本次仅补充文档说明

v1.6.7 (2026-01-23)

新增约束实现：

- **角度比例约束：**支持 `angle(AOB)=k*angle(BOC)` 语法
 - 新增 `STAR` token 类型用于乘法运算符
 - 新增 `AngleRatioConstraint` AST 节点和约束类
 - Parser 扩展：`parse_angle_constraint()` 支持解析 `NUMBER*angle(...)` 表达式
 - Constraint 实现：`compute_error()` 基于角度值比例计算误差
 - 适用场景：表达角度间的倍数关系（如 $\angle AOB = (1/3)\angle BOC$ ）

使用说明：

- 比例系数 k 必须是数值字面量（不支持符号变量）
- 分数需预先计算为小数： $(1/3) \rightarrow 0.3333$
- 示例：`angle(AOB)=2*angle(BOC)`, `angle(CAD)=0.5*angle(BAC)`

v1.6.6 (2026-01-23)

新增约束实现：

- **弧中点约束：**`arc_midpoint(D, BC, circle_O, arc_type='minor'|'major')`
 - 支持劣弧/优弧中点的精确表达
 - 约束条件： $|OD| = |OB|$ 且 $\angle BOD = \angle COD$ （圆心角相等）
 - 新增 `ARC_MIDPOINT` token、`ArcMidpointConstraint` AST 节点和约束类
 - 智能初始化：`_infer_from_arc_midpoint()` 根据弦中点和垂直方向自动推断初始位置
 - 默认 `arc_type='minor'`（劣弧），可选 `'major'`（优弧）

拓扑排序规范增强：

- **Layer 2 vs Layer 7 区分：**在 17.1.1 节添加详细说明
 - Layer 2（图形声明）：声明几何对象的**存在性**（如 `circle(0)`, `triangle(A,B,C)`）
 - Layer 7（几何关系）：描述已存在对象间的**关系**（如 `inscribed(...)`, `tangent(...)`）
 - 关键原则：先声明对象，再描述关系

语义展开简化：

- **bisect_angle** 展开简化为仅 `bisector(l, ABC)`，去掉匿名点 P
- **bisector** 语义明确蕴含 `on(B, l)`（角平分线必过顶点）

类型语义澄清：

- 新增"线段与直线的语义区分"表，明确 `on(P, AB)` 为线段语义
- 明确"直线上的点"应使用 `collinear(A, P, B)`
- 明确 `parallel/perpendicular` 中 SEGMENT 解释为"过两点的直线"

Canonicalization 增强：

- 新增 17.2.1 节"等价写法归一化"规则表
- 明确 `equal_angle` 为"无向内角相等"语义
- V11 隐含约束纳入归一化流程

v1.6.6 (2026-01-20)

坐标轴约束语法文档化：

- **文档化 `on(P, axis_x)` 和 `on(P, axis_y)`：**将已实现的坐标轴约束语法添加到 DSL 规范

语法：

```
on(P, axis_x)           # 点 P 在 x 轴上 (P.y = 0)
on(P, axis_y)           # 点 P 在 y 轴上 (P.x = 0)
```

约束说明：

- `on(P, axis_x)` 约束点 P 的 y 坐标为 0

- `on(P, axis_y)` 约束点 P 的 x 坐标为 0
- 通常与 `coordinate_system(0)` 配合使用
- 也可独立使用，以默认原点 (0, 0) 为参考

Solver 实现：

- `PointOnAxisConstraint` - 坐标轴上点约束（已在 v1.5.0 实现）
- Parser 识别 `axis_x` / `axis_y` 关键字
- Extractor 生成 `PointOnAxisConstraint`

对应中文：

- 「点 A 在 x 轴上」→ `on(A, axis_x)`
- 「点 B 在 y 轴上」→ `on(B, axis_y)`

v1.6.5 (2026-01-22)

弧上点约束 (新增)：

- 新增 `on(E, arc_BC)`：约束点 E 在弧 BC 上

语法：

```
arc(B, C, circle_0)    # 声明弧 BC（圆 O 上从 B 到 C 的弧）
on(E, arc_BC)          # 点 E 在弧 BC 上
```

约束说明：

- `arc_BC` 引用格式表示从 B 到 C 的弧（逆时针方向的劣弧）
- 使用前必须先声明 `arc(B, C, circle_0)` 以关联圆心
- 约束等价于：点在圆上 + 角度在弧范围内

Solver 实现：

- 新增 `PointOnArcConstraint` - 弧上点约束
- Parser 识别 `arc_` 前缀的 `location_type`
- Extractor 跟踪弧声明并生成相应约束
- NumericSolver 支持弧上点的智能初始化

v1.6.4 (2026-01-21)

角度三角函数值约束 (新增):

- 新增 `sin(ABC)=value`: 约束角度 ABC 的正弦值
- 新增 `cos(ABC)=value`: 约束角度 ABC 的余弦值
- 新增 `tan(ABC)=value`: 约束角度 ABC 的正切值

语法:

```
sin(ABC)=0.5           # ∠ABC 的正弦值为 0.5
cos(ABC)=0.866         # ∠ABC 的余弦值为 √3/2
tan(ABC)=1              # ∠ABC 的正切值为 1
```

Solver 实现:

- 新增 `AngleSinConstraint` - 正弦值约束
- 新增 `AngleCosConstraint` - 余弦值约束
- 新增 `AngleTanConstraint` - 正切值约束

v1.6.3 (2026-01-20)

REGION_REF 扩展:

- 新增 `RECTANGLE_REF`: 支持 `rectangle_ABCD` 格式的矩形区域引用
- 新增 `PARALLELOGRAM_REF`: 支持 `parallelogram_ABCD` 格式的平行四边形区域引用
- 新增 `SQUARE_REF`: 支持 `square_ABCD` 格式的正方形区域引用
- 新增 `QUADRILATERAL_REF`: 支持 `quadrilateral_ABCD` 格式的四边形区域引用
- 新增 `POLYGON_REF`: 支持 `polygon_ABC...` 格式的任意多边形区域引用

inside/outside 约束增强:

- `inside(P, rectangle_ABCD)` - 点 P 在矩形 ABCD 内部
- `outside(P, rectangle_ABCD)` - 点 P 在矩形 ABCD 外部
- 支持所有四边形类型 (rectangle、parallelogram、square、quadrilateral)
- 支持任意多边形 (polygon)

Solver 实现:

- 新增 `InsideQuadrilateralConstraint` - 四边形内部软约束
- 新增 `OutsideQuadrilateralConstraint` - 四边形外部软约束
- 新增 `InsidePolygonConstraint` - 多边形内部软约束

- 新增 `OutsidePolygonConstraint` - 多边形外部软约束
- 新增 `OutsideTriangleConstraint` - 三角形外部软约束（补充缺失）

v1.6.2 (2026-01-16)

Validity Rules 完善:

- **V4 细化:** 几何度量角度（内角）必须在 $(0, 180]$ 范围内，明确"内角"语义
- **V4a 新增:** 变换操作角度允许负值（单位：度），负值表示顺时针
- **V11 新增:** `tangent(A, circle_0)` 隐含 `on(A, circle_0)`，validator 支持 strict/autocorrect 双模式

v1.6.1 (2026-01-16)

EBNF 完整性修复:

- **补充 inscribed/circumscribed/circumcircle:** 在 `relation_constraint` 中添加内接、外接圆关系的产生式
- **补充 arc_decl:** 在 `decl_stmt` 中添加弧声明 `arc(POINT, POINT, CIRCLE_REF)`
- **修正 POINT 终结符:** 将 `[A-Z] [A-Z0-9]*` 改为 `[A-Z] [0-9]*`，避免与 SEGMENT 产生歧义
- **新增 RAY_REF 终结符:** 定义射线引用格式 `ray_POINT POINT`
- **新增 BISECTOR_REF 终结符:** 定义角平分线引用格式 `bisector_ANGLE`
- **新增 SECTOR_REF 终结符:** 定义扇形引用格式 `sector_POINT POINT POINT`
- **更新 REGION_REF:** 使用新定义的 SECTOR_REF
- **更新 LOCATION:** 使用新定义的 RAY_REF

语法一致性修复:

- **统一 sector 语法:** 将 14.3.3 节的 `sector(P, O, A, B)` 改为 `inside(P, sector_OAB)`，与 EBNF 3 参数定义一致
- **更新对照表:** 修正 14.7 节中扇形相关的 DSL 转换

语义清晰度改进:

- **明确 fold 与 reflect 关系:** 在 12.2 节添加两者参数顺序差异和等价关系说明
- **添加操作/宏命名规范:** 在第 8 章添加 8.0 节，说明 `draw_*` 操作指令与宏指令的参数顺序设计原则
- **重写 tangent 语义:** 在 5.7 节明确 `tangent(POINT, CIRCLE)` 表示"该点是切点"
- **添加 tangent_point/tangent_line 参数说明:** 在 13.6 节说明参数顺序设计原则

文档细节修正：

- **明确 axis_x/axis_y 状态：**在 13.1 节标注为辅助指令，说明通常无需显式使用
- **完善 ANGLE 注释：**在 EBNF 中添加 ANGLE 在连写和逗号分隔两种上下文的说明
- **修正 isosceles_right 描述：**将 15.5 节"直角在 A"改为"AB=AC， $\angle A=90^\circ$ "

v1.6.0 (2025-12-28)

EBNF 语法一致性修复：

- **NUMBER 支持负数：**NUMBER 更新为 `SIGNED_NUMBER ::= "-"? [0-9]+ ("." [0-9]+)?`
- **新增 VALUE 类型：**`VALUE ::= SIGNED_NUMBER | SYMBOL`，统一函数参数的数值/符号表示
- **统一 ray 格式：**移除 `ray(AB)` 格式，仅保留 `ray(A, B)` 作为唯一有效格式
- **扩展 intersection：**新增 `intersection(P, LINE, CIRCLE_REF)` 和 `intersection(P, CIRCLE_REF, CIRCLE_REF)` 支持
- **扩展 tangent：**新增 `tangent(LINE, CIRCLE_REF)` 表示直线与圆相切
- **移除 circle_coord：**删除未实现的 `circle_coord` 描述，改用 `circle(0, r=N) + point_coord(0, x, y)` 组合

新增章节：

- **第 10.4 节 - 角度单位约定：**明确几何约束使用度 (degree)，三角函数相位使用弧度 (radian)
- **第 17 章 - 标准化与规范化 (Canonicalization)：**定义 DSL 语句拓扑排序规则、宏展开规则、点名标准化

章节编号调整：

- 原第 17 章 (标注示例) → 第 18 章
- 原第 18 章 (版本管理) → 第 19 章

v1.5.3 (2025-12-07)

语法一致性强化：

- **移除旧版 rotate(P, center=0, angle=N) 语法** (无 source= 参数)
- **唯一有效格式：**`rotate(D, source=C, center=0, angle=N)`
- **fold(C, C', EF)** 现在等价于 `reflect(C', C, EF)`，不再产生无效的 SegmentDecl

新增约束实现：

- `ConcyclicConstraint` - 四点共圆约束（4x4 行列式条件）
- `EqualAngleConstraint` - 等角约束（余弦相等条件）
- `SegmentLengthRatioConstraint` - 线段长度比约束（ $\text{ratio}(AB, CD)=2$ ）

修复缺失的访问方法：

- 新增 `visit_ray_decl` - 射线声明处理
- 新增 `parse_ray_decl` - 射线解析方法

保留关键字完善：

- 新增 ~20 个缺失的保留关键字到 `RESERVED_KEYWORDS`
- 包括：`ray`, `polygon`, `rectangle`, `parallelogram`, `square`, `concyclic`, `fold`, `isosceles` 等

v1.5.0 (2025-12-06)

新增符号参数支持：

- 函数声明支持符号变量作为参数：`parabola(p, a, b, c)`、`hyperbola(h, k)`、`line_function(l, k1, b)`
- 新增 AST 节点类型：`ValueNode`（基类）、`NumberValue`（数值）、`SymbolicValue`（符号变量）
- Parser 新增 `parse_value()` 方法，支持解析数字或符号变量
- Extractor 新增 `_resolve_value()` 方法，自动解析符号值为默认数值（用于可视化）
- 支持符号上下文设置：`extractor.symbol_context["k"] = 5.0`

新增几何约束：

- `trisection(P, AB, near=A)` - 三等分点约束
- `perpendicular_bisector(M, AB)` - 垂直平分线约束
- `median(M, A, BC)` - 中线约束
- `similar(ABC, DEF)` - 相似三角形约束

新增函数几何约束：

- `coordinate_system(O)` - 坐标系声明，自动设置原点 $O=(0,0)$
- `function_point(P, f)` - 点在函数图像上
- `axis_intersection(A, f, x/y)` - 函数与坐标轴交点

- `function_intersection(P, f, g)` - 两函数交点
- `tangent_point(C, l, f)` - 切点约束
- `tangent(A, circle_0)` - 过点的圆切线

C++ 后端扩展:

- 新增 `TANGENT_POINT_CIRCLE` 约束实现
- 新增 `TRISECTION` 约束实现
- 新增 `PERPENDICULAR_BISECTOR` 约束实现
- 新增 `SIMILAR_TRIANGLES` 约束实现

AST Round-trip 支持:

- 所有 AST 节点实现 `to_dsl()` 方法, 支持 AST → DSL 序列化
- `NumberValue.to_dsl()` 对整数输出整数格式 (如 `2` 而非 `2.0`)
- `SymbolicValue.to_dsl()` 输出符号名称

数据集修复:

- 修复 8 个语法错误的标注文件
- 修复 4 个使用 `axis_x/axis_y` 的文件 (改用共线约束)
- 解析成功率: 70.1% → 100%
- 约束提取成功率: 70.1% → 100%

测试覆盖:

- 新增 11 个符号参数测试用例
- 总测试数: 151 个 (全部通过)

v1.4.1 (2025-12-06)

新增阴影区域标记:

- 新增 `shaded(REGION_REF)` 语法, 用于标记阴影/填充区域
- 支持三角形、扇形、圆等区域的阴影标记
- 在 `region_stmt` 产生式中添加相应定义

扩展旋转变换语法:

- 扩展 `rotate` 语法, 新增 `source` 参数
- 支持 `rotate(D, source=C, center=0, angle=60)` 格式
- 明确指定旋转源点和结果点的对应关系

v1.4.0 (2025-12-03)

新增模糊语义消歧指令：

- 线段方向消歧：`direction(P, AB, towards=A/B)`
- 延长线方向消歧：`on_extended(P, AB, towards=A/B)`
- 区域内部/外部：`inside(P, region), outside(P, region)`
- 扇形区域：`sector(P, O, A, B)`
- 半平面：`half_plane(P, AB, side=left/right)`
- 角平分线消歧：`on(P, bisector_ABC, near=B)`
- 任选点：`choose(P, on=AB), choose(P, in=triangle_ABC)`

新增终结符：

- `TRIANGLE_REF`：三角形区域引用
- `REGION_REF`：区域引用（三角形/圆/扇形）
- `SIDE`：半平面侧向（left/right）

语法更新：

- EBNF 语法定义新增 `disambig_stmt` 产生式
- 新增第 14 章：模糊语义消歧指令

设计目的：

- 解决 LLM 从中文到 DSL 解析时的歧义问题
- 为 Solver 提供明确的初始点位置提示
- 支持区域约束的 soft constraint 处理

v1.3.0 (2025-12-02)

新增函数类型：

- 新增幂函数：`power_function(f, n) →  $y = x^n$`
- 新增指数函数：`exp_function(f, a) →  $y = a^x$`
- 新增对数函数：`log_function(f, a) →  $y = \log_a(x)$`
- 新增三角函数：`sin_function, cos_function, tan_function, cot_function, sec_function, csc_function`
- 新增反三角函数：`arcsin_function, arccos_function, arctan_function, arccot_function`
- 新增通用函数表达式：`function_expr(f, "expression")` 支持复合函数

新增函数变换：

- 平移变换：`translate_function(f, g, dx, dy)`
- 伸缩变换：`scale_function(f, g, sx, sy)`
- 对称变换：`reflect_function(f, g, axis)`

新增函数与几何交互：

- 切线：`tangent_line(l, f, P)`
- 渐近线：`asymptote(l, f)`

语法更新：

- EBNF 语法定义新增函数相关产生式
- 新增终结符：`STRING, AXIS, FUNC_REF`

v1.2.0 (2025-11-13)

新增功能：

- 新增坐标系与函数图像章节（第 13 节）
- 新增坐标系声明：`coordinate_system, axis_x, axis_y`
- 新增坐标点声明：`point_coord`
- 新增函数图像声明：`line_function, hyperbola, parabola`
- 新增函数与几何交互：`function_point, function_intersection, axis_intersection, tangent_point`
- 新增坐标系中的圆：`circle_coord`

结构调整：

- 章节编号顺延，原 13-16 节变为 14-17 节

v1.1.0 (2025-10-25)

新增功能：

- 新增操作型构造指令章节（`connect, extend, draw_perpendicular, draw_parallel, bisect_angle, draw_altitude, draw_median`）
- 新增宏指令展开规则（`median, altitude, angle_bisector, perpendicular_bisector`）
- 新增几何关系：`collinear, concyclic, congruent, similar, bisector`
- 新增中文表达对照表章节

- 新增 EBNF 形式化语法定义
- 新增完整标注示例
- 新增版本管理机制

增强功能：

- 支持多点声明语法 `points(A, B, C, D)`
- 支持直线命名 `line(l, A, B)`
- 支持圆半径声明 `circle(O, r=5)` 和 `circle(O, radius=AB)`
- 增强特殊三角形宏的展开规则文档

结构调整：

- 重组章节顺序，使结构更清晰
- 将图形结构与宏指令分离为独立章节

v1.0.0 (2025-10-20)

- 初始版本发布
- 定义基础元素、几何关系、图形结构
- 定义圆与弧、角度与长度表达
- 定义点的位置关系、几何变换与折叠操作
- 定义合法性规则

注意：本规范为 NeoGeo 项目的正式 DSL 定义文档，所有标注员和模型训练过程必须严格遵循本规范。如有疑问或发现规范缺陷，请联系项目维护者陈铄涵 chenshuohan60@gmail.com。